

Ruben Apablaza Muñoz

PIPE

y la puerta del
reino de Linux

HTB

Linux Fundamentals



— 0Zone —

⚠ **AVISO LEGAL Y DESCARGO DE RESPONSABILIDAD** ⚠

Este documento tiene fines **estrictamente educativos y de formación profesional** en el área de la ciberseguridad. El autor no se hace responsable por el mal uso de la información, scripts o comandos aquí descritos.

- **Entornos de prueba**

Se recomienda ejecutar este laboratorio exclusivamente en entornos controlados (Máquinas Virtuales o Contenedores). Nunca ejecute scripts o comandos desconocidos en servidores de producción sin antes probarlos y entender su funcionamiento.

- **Responsabilidad del usuario**

El uso de herramientas de nivel administrativo (sudo) conlleva riesgos. El lector es el único responsable de cualquier pérdida de datos o interrupción del servicio que pudiera derivarse de la aplicación de este manual.

- **Ética profesional**

Las técnicas de análisis aquí descritas deben ser utilizadas siempre bajo un marco legal y con la autorización debida.

Contenido

Introducción	4
Linux Fundamental HTB.....	4
Sección 2: The Shell	6
Resumen rápido del capítulo.	6
Información del sistema	6
Conexión via SSH.....	7
QUESTION 1.....	11
QUESTION 2	13
QUESTION 3.....	14
QUESTION 4.....	17
QUESTION 5.....	18
QUESTION 6.....	19
Sección 3: Workflow.....	20
Navigation	20
QUESTION 1	20
QUESTION 2	23
Working with Files and Directories.....	28
QUESTION 1.....	28
QUESTION 2	31
Find Files and Directories.....	32
QUESTION 1	32
QUESTION 2	40
QUESTION 3.....	42
File Descriptors and Redirections	44
QUESTION 1	44
QUESTION 2	45
Filter Contents.....	50
QUESTION 1	52
QUESTION 2	58

QUESTION 3.....	60
Regular Expressions.....	64
Permission Management.....	66
Sección 4: System Management.....	67
User Management.....	67
QUESTION 1.....	67
QUESTION 2.....	70
QUESTION 3.....	72
Service and Process Management.....	73
QUESTION 1.....	73
Task Scheduling.....	74
QUESTION 1.....	74
Network Services.....	80
QUESTION 1.....	80
QUESTION 2.....	82
File System Management.....	83
Question 1.....	84
Conclusión.....	85
¿Hemos terminado?.....	85

Introducción

Linux Fundamental HTB

Cuando Pipe (|) estaba recién comenzando a aprender Linux, encontró un sinfín de recursos: videotutoriales, libros de Linux para hackers, la Academia Red Hat y muchos otros. Pero siempre tenía esa sensación constante de inseguridad; no sabía si realmente era capaz de resolver ciertos problemas por su cuenta. Muchos de esos cursos te llevan de la mano; lo cual no está mal, ya que ayudaron a construir los cimientos, pero a veces nos lanzamos como caballo loco a ejecutar scripts y comandos sin entender realmente qué está sucediendo por abajo. Por ejemplo, él sabía de la existencia de systemd, pero no terminaba de comprender su relación con systemctl, o por qué utilizábamos apt en vez de dpkg.

Necesitaba volver a las bases para ordenar todo ese conocimiento adquirido desde diversas fuentes; a Pipe le gusta entender REALMENTE lo que está haciendo, más que repetir lo que otra persona hace. Ahora que lo hizo, muchos de esos otros recursos tienen mucho más sentido.

En esa búsqueda dió con el curso fundamentals de Linux de Hack The Box (HTB). Si bien es más que sabido que HTB tiene una tendencia clara hacia el red team, y su teoría no pretende reinventar la rueda, lo cierto es que sus cuestionarios le obligaron a repensar las bases.

Con esa premisa nace este documento: un recorrido que convierte unas pocas preguntas en un barrido profundo para responder el porqué y el cómo se llega a la respuesta desde una lógica personal. Al adentrarse por las puertas del reino de Linux, Pipe aprendió varias cosas que no sabía, aclaró varias que creía saber y confirmó mucho de lo que ya dominaba.

¿Con esto puedo decir que soy un "experto" en Linux?

Ciertamente no. Pero la seguridad que ha ganado le ha quitado esa sensación de vacío y le da el respaldo de unas bases sólidas para seguir aprendiendo.

Esta no es una invitación a hacer el curso de HTB; es un llamado a que cualquiera que se sienta así construya su propio camino. Y con esto me refiero a documentar, a plantearse dudas más allá de solo colgar una captura de pantalla con la respuesta correcta sin que nadie entienda cómo se llegó a esa conclusión... Creo que ese fue el mejor ejercicio: obligarse a escribir como si alguien más tuviese que entenderlo.

¿Recomiendo este curso? Ciertamente que sí, como también cualquier recurso que nos ayude a entender los fundamentos.

Así que si no lo has hecho, acompañemos a Pipe (|) y sus amigos a cruzar la puerta del reino de Linux y consolidar sus fundamentos recorriendo el camino del curso de HTB Linux Fundamentals.



Sección 2: The Shell

Resumen rápido del capítulo.

Información del sistema

Se hace referencia a la importancia de comandos como `-h`, `--help` y `man`, este último resulta muy útil para conocer las opciones de ejecución de los diversos comandos en Linux. **Conviene intrusear con ellos en la terminal.**

Command	Description
<code>whoami</code>	Displays current username.
<code>id</code>	Returns users identity
<code>hostname</code>	Sets or prints the name of current host system.
<code>uname</code>	Prints basic information about the operating system name and system hardware.
<code>pwd</code>	Returns working directory name.
<code>ifconfig</code>	The ifconfig utility is used to assign or to view an address to a network interface and/or configure network interface parameters.
<code>ip</code>	Ip is a utility to show or manipulate routing, network devices, interfaces and tunnels.
<code>netstat</code>	Shows network status.
<code>ss</code>	Another utility to investigate sockets.
<code>ps</code>	Shows process status.
<code>who</code>	Displays who is logged in.
<code>env</code>	Prints environment or sets and executes command.
<code>lsblk</code>	Lists block devices.
<code>lsusb</code>	Lists USB devices
<code>lsdf</code>	Lists opened files.
<code>lspci</code>	Lists PCI devices.

Conexión via SSH

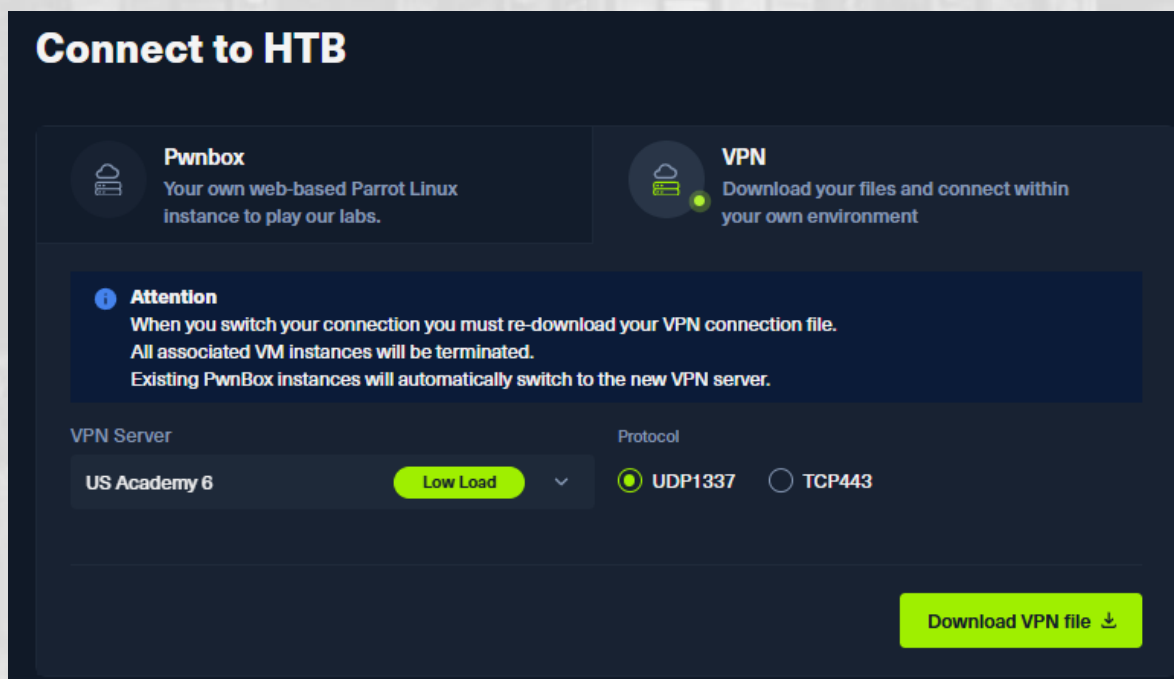
En esta sección también te enseñan a conectar vía SSH para el desarrollo de los quiz que la verdad es donde más se aprende.

Un comando útil para desarrollar el quiz es el comando **uname** que sirve para imprimir información del sistema. Es indispensable para responder el quiz, si no lo conoces te recomiendo utilizarlo en conjunto con **man**. Con eso se tiene acceso a conocer información específica que servirá para responder.

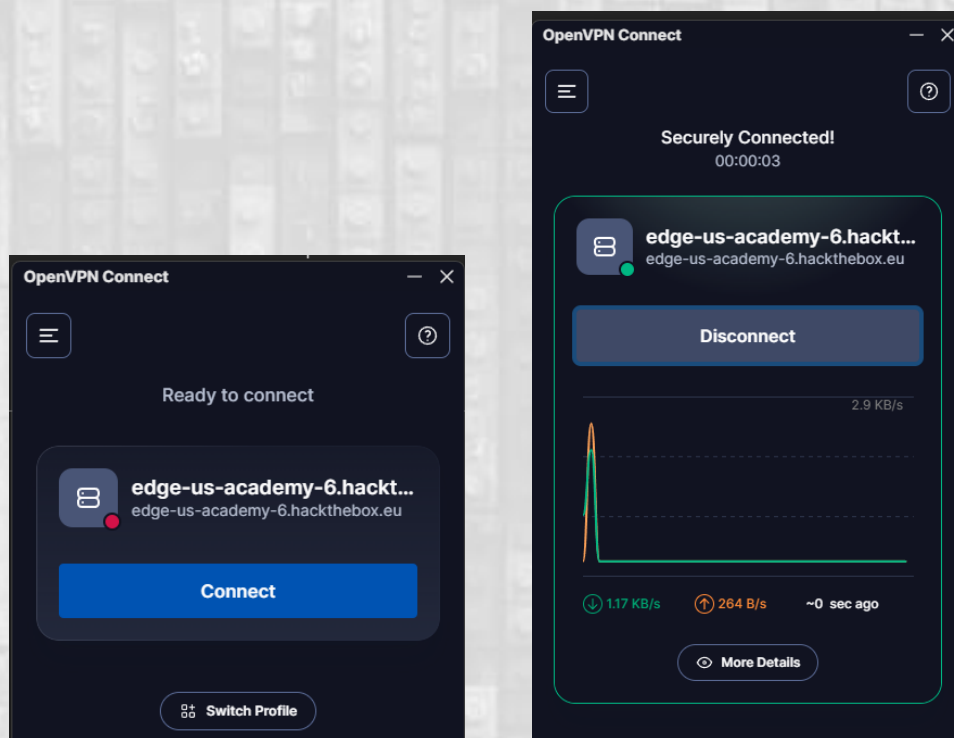
man uname

Si bien el sitio de HTB da la opción de abrir una terminal virtual, es una opción bastante limitada ya que permite abrir una sesión cada 24 horas por lo que es recomendable hacer la conexión por vpn.

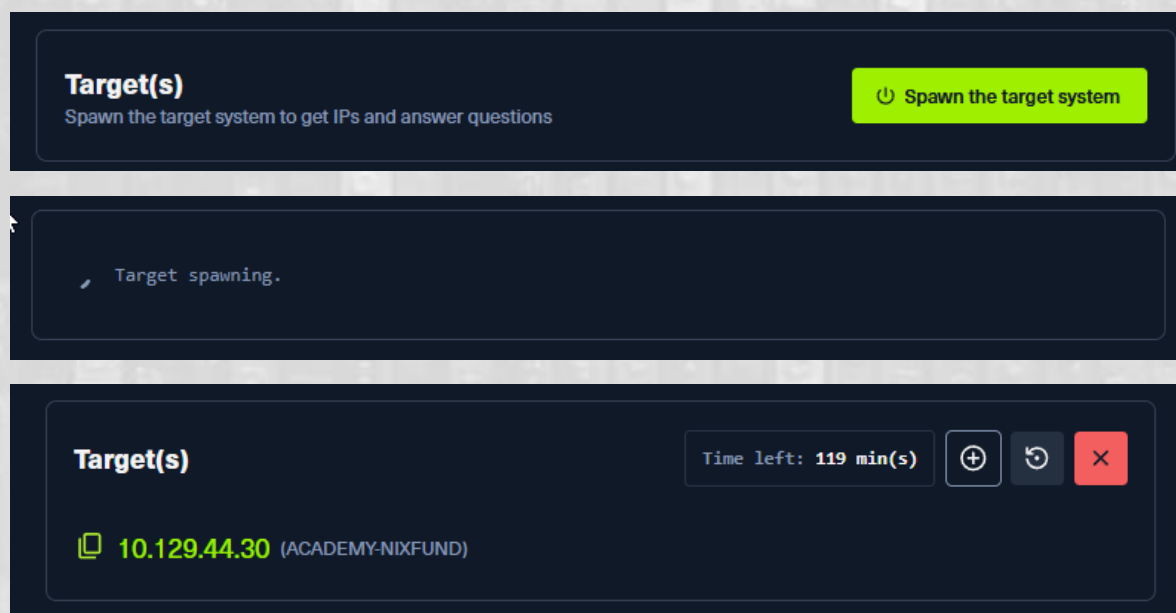
Para esto podemos utilizar **OpenVPN**; desde la página de HTB descargamos el archivo VPN y lo abrimos en nuestro programa y con eso ya tendremos una conexión segura.



Pipe y la puerta del reino de Linux | Sección 2: The Shell

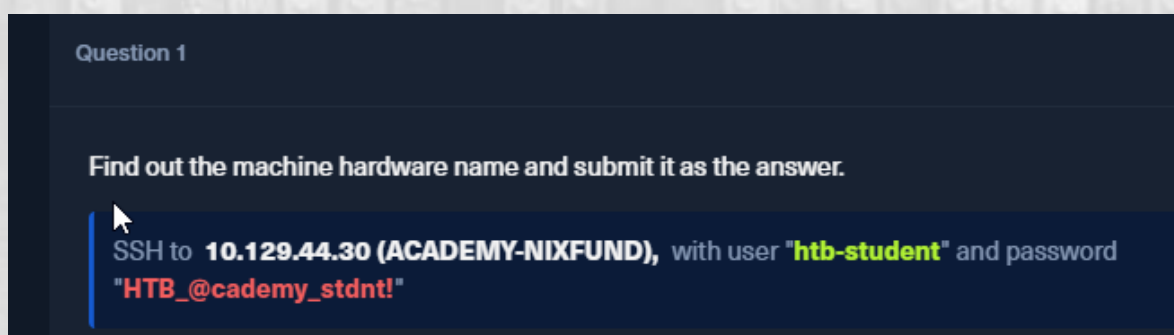


Una vez conectados a la vpn haremos click en **Spawn the target system**, una vez iniciada la maquina nos entregará una dirección ip para conectarnos por SSH.

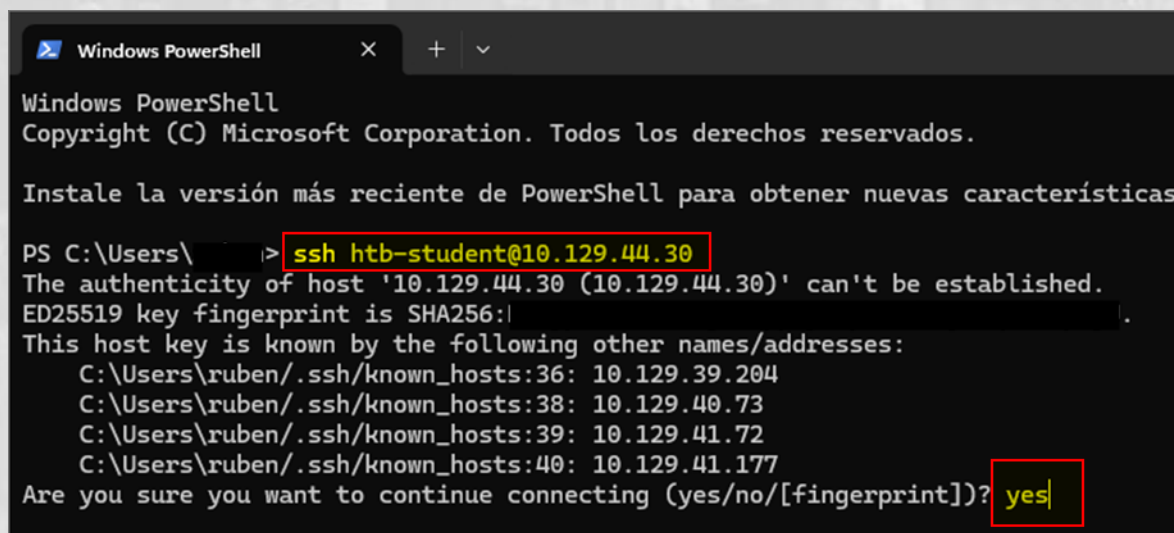


Pipe y la puerta del reino de Linux | Sección 2: The Shell

En la primera pregunta nos entregan las credenciales para conectar via SSH. Un pequeño tip para rellenar los campos de nuestra conexión en nuestra terminal es hacer click sobre los campos resaltados como la dirección ip que nos entregó previamente y el usuario y la password, con esto los valores quedan copiados al portapapeles en este caso de Windows y puedo simplemente darle a CTRL + V para pegarlos y así evitar posibles errores de tipeo, sobre todo con la contraseña que por seguridad no se muestra mientras tipeamos.



Ya con las credenciales abrimos una terminal para conectar via SSH, e ingresamos, cuando nos pida confirmación para continuar escribimos "yes"



Pipe y la puerta del reino de Linux | Sección 2: The Shell

A continuación solicitará la password

```
PS C:\Users\ruben> ssh htb-student@10.129.44.30
The authenticity of host '10.129.44.30 (10.129.44.30)' can't be established.
ED25519 key fingerprint is SHA256:
This host key is known by the following other names/addresses:
  C:\Users\ruben/.ssh/known_hosts:36: 10.129.39.204
  C:\Users\ruben/.ssh/known_hosts:38: 10.129.40.73
  C:\Users\ruben/.ssh/known_hosts:39: 10.129.41.72
  C:\Users\ruben/.ssh/known_hosts:40: 10.129.41.177
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.44.30' (ED25519) to the list of known hosts.
htb-student@10.129.44.30's password:
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.15.0-123-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage
```

Y con esto ya estamos conectados a la máquina de HTB

```
System information as of Fri Jun 26 15:33:49 UTC 2026

System load:  0.38           Processes:            155
Usage of /:   51.0% of 6.76GB Users logged in:       0
Memory usage: 20%           IP address for ens192: 10.129.44.30
Swap usage:   0%

 * Canonical Livepatch is available for installation.
   - Reduce system reboots and improve kernel security. Activate at:
     https://ubuntu.com/livepatch

0 packages can be updated.
0 updates are security updates.

Last login: Wed Sep 23 22:09:41 2020 from 10.10.14.6
htb-student@nixfund:~$
```

Con CTRL + L limpiamos la terminal y ya podemos comenzar.

QUESTION 1

Find out the machine hardware name and submit it as the answer.

Para responder al nombre del hardware de la maquina utilizaremos el comando **uname**, el problema es que con eso solo obtenemos el nombre del sistema, así que avanzaremos hacia el comando **uname -a** que nos entrega más información del sistema.

```
htb-student@nixfund:~$ uname
Linux
htb-student@nixfund:~$ uname -a
Linux nixfund 4.15.0-123-generic #126-Ubuntu SMP Wed Oct 21 09:40:11 UTC 2020 x86_64 x86_64 x86_64 GNU
/Linux
htb-student@nixfund:~$ |
```

Pero debido a que queremos una respuesta especifica y no sabemos cuál es la opción que da el nombre del hardware utilizaremos **man uname**.



Pipe y la puerta del reino de Linux | Sección 2: The Shell

```
UNAME(1)                                User Commands                                UNAME

NAME
    uname - print system information

SYNOPSIS
    uname [OPTION]...

DESCRIPTION
    Print certain system information.  With no OPTION, same as -s.

    -a, --all
        print all information, in the following order, except omit -p and -i if unknown:

    -s, --kernel-name
        print the kernel name

    -n, --nodename
        print the network node hostname

    -r, --kernel-release
        print the kernel release

    -v, --kernel-version
        print the kernel version

    -m, --machine
        print the machine hardware name

    -p, --processor
        print the processor type (non-portable)

    -i, --hardware-platform
        print the hardware platform (non-portable)

    -o, --operating-system
        print the operating system

    --help display this help and exit

    --version
        output version information and exit

AUTHOR
    Written by David MacKenzie.

REPORTING BUGS
    GNU coreutils online help: <http://www.gnu.org/software/coreutils/>
    Report uname translation bugs to <http://translationproject.org/team/>
```

Podemos ver que la opción **-m** nos dará la respuesta que estamos buscando. Esta es la importancia de los comandos de ayuda como **man** y **help**, te ayudan a buscar caminos. Vamos a probar.

```
htb-student@nixfund:~$ uname
Linux
htb-student@nixfund:~$ uname -a
Linux nixfund 4.15.0-123-generic #126-Ubuntu SMP Wed Oct 21 09:40:11 UTC 2020 x86_64 x86_64 x86_64 GNU
/Linux
htb-student@nixfund:~$ man uname
htb-student@nixfund:~$ uname -m
x86_64
htb-student@nixfund:~$ |
```

Y podemos ver el resultado correcto

Pipe y la puerta del reino de Linux | Sección 2: The Shell

Ingresamos la respuesta y voila.

Question 1

XP +20 ^

Find out the machine hardware name and submit it as the answer.

SSH to **10.129.44.30 (ACADEMY-NIXFUND)**, with user **"htb-student"** and password **"HTB_@cademy_stdnt!"**

x86_64

Hint

QUESTION 2

What is the path to htb-student's home directory?

Para responder a esta pregunta utilizaremos uno de los primeros comandos que se aprende en todos los cursos de Linux, básicamente sirve para saber **DONDE ESTAMOS**, me refiero a **pwd**

```
htb-student@nixfund:~$ pwd
/home/htb-student
htb-student@nixfund:~$
```

Este comando nos entrega la ruta donde estamos.

Question 2

+1 XP +20 ^

What is the path to htb-student's home directory?

/home/htb-student

QUESTION 3

What is the path to the htb-student's mail?

En este punto algunos puedan pensar que quizás se pueda utilizar **locate MAIL** o **locate mail**. El problema con esto es que **locate**, busca en todo el disco archivos que lleven las palabras **MAIL** o **mail** en su nombre (recordar que Linux es Case sensitive y por lo tanto hay diferencias).

```
htb-student@nixfund:~$ locate MAIL
/usr/share/doc/wget/MAILING-LIST
```

```
htb-student@nixfund:~$ locate mail
/etc/mailcap
/etc/mailcap.order
/etc/apparmor.d/abstractions/ubuntu-console-email
/etc/apparmor.d/abstractions/ubuntu-email
/etc/apparmor.d/abstractions/user-mail
/etc/apparmor.d/abstractions/ubuntu-browsers.d/mailto
/etc/dovecot/conf.d/10-mail.conf
/etc/dovecot/conf.d/15-mailboxes.conf
/etc/dovecot/conf.d/auth-vpopmail.conf.ext
/lib/modules/4.15.0-122-generic/kernel/drivers/mailbox
/lib/modules/4.15.0-122-generic/kernel/drivers/leds/leds-clevo-mail.ko
/lib/modules/4.15.0-122-generic/kernel/drivers/mailbox/mailbox-altera.ko
/lib/x86_64-linux-gnu/security/pam_mail.so
/snap/core/10126/etc/mailcap
/snap/core/10126/etc/mailcap.order
/snap/core/10126/etc/apparmor.d/abstractions/ubuntu-console-email
/snap/core/10126/etc/apparmor.d/abstractions/ubuntu-email
/snap/core/10126/etc/apparmor.d/abstractions/user-mail
/snap/core/10126/etc/apparmor.d/abstractions/ubuntu-browsers.d/mailto
/snap/core/10126/lib/systemd/system/mail-transport-agent.target
/snap/core/10126/lib/x86_64-linux-gnu/security/pam_mail.so
/snap/core/10126/usr/bin/mail-lock
/snap/core/10126/usr/bin/mail-touchlock
/snap/core/10126/usr/bin/mail-unlock
/snap/core/10126/usr/bin/run-mailcap
/snap/core/10126/usr/lib/gnupg/gpgkeys_mailto
/snap/core/10126/usr/lib/mime/mailcap
/snap/core/10126/usr/lib/python3.5/email
/snap/core/10126/usr/lib/python3.5/mailbox.py
/snap/core/10126/usr/lib/python3.5/mailcap.py
/snap/core/10126/usr/lib/python3.5/__pycache__/mailbox.cpython-35.pyc
/snap/core/10126/usr/lib/python3.5/__pycache__/mailcap.cpython-35.pyc
/snap/core/10126/usr/lib/python3.5/email/_init_.py
```

Pipe y la puerta del reino de Linux | Sección 2: The Shell

Lo que nos piden en la pregunta, es la ruta del correo del usuario. El sistema guarda esa ruta dentro de una variable, en este caso `$MAIL` y aquí es donde nos aparece otro comando, `echo`.

El comando `echo` sirve para pedir a la terminal que muestre el valor de variables de entorno, como en este caso la variable es `mail`, el comando completo será `echo $MAIL`

Pero... WAIT... ¿Por qué si la variante es mail el comando lleva \$MAIL? ¿De dónde sale el "\$" y las MAYUSCULAS?

El signo `$` es el activador que le dice a la terminal *"lo que viene no es una palabra NORMAL, es una VARIABLE (caja con información)"*

Dicho de otro modo:

- Si **NO** pones el `$`:

`echo MAIL` . La terminal entiende la palabra de forma literal. Solo va a escribir en la pantalla la palabra **MAIL**.

```
htb-student@nixfund:~$ echo MAIL
MAIL
htb-student@nixfund:~$ echo mail
mail
htb-student@nixfund:~$ |
```

- Si **SÍ** pones el `$`:

`echo $MAIL` . El `$` transforma la palabra en una variable. La terminal entiende: *"Ah, quieres que abra la caja con información (variable) llamada MAIL y te muestre lo que hay dentro"*. Y entonces nos muestra la ruta (ej. `/var/mail/htb-student`).

```
htb-student@nixfund:~$ echo $mail
/var/mail/htb-student
htb-student@nixfund:~$ echo $MAIL
/var/mail/htb-student
htb-student@nixfund:~$ |
```


Ya ... pero ¿y las mayúsculas?

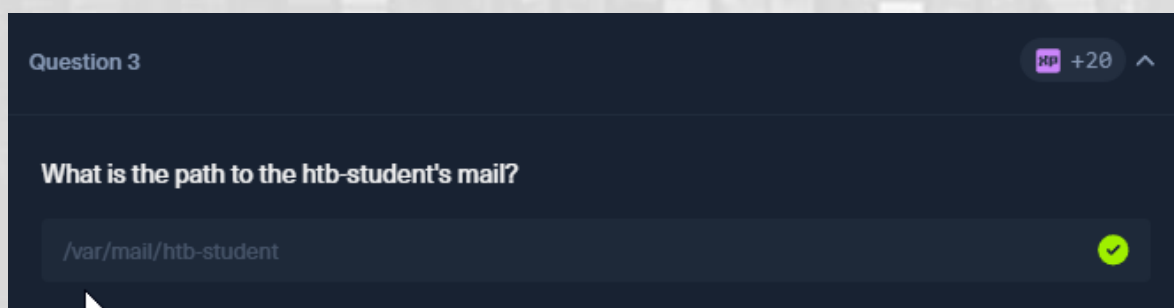
Porque **Linux distingue entre mayúsculas y minúsculas** como se ha señalado previamente. Para el sistema, **\$MAIL** (en mayúsculas) y **\$mail** (en minúsculas) son dos cosas completamente distintas.

La explicación rápida:

- **\$MAIL** : Es la variable oficial que el sistema Linux crea automáticamente para guardar la ruta del correo.
- **\$mail** : No existe. Es una caja vacía.

Como se ve en la imagen anterior, si ejecutamos **echo \$mail**, la terminal buscará una variable en minúsculas, no encontrará nada y nos devolvió una línea en blanco.

Recordar esta **regla de oro en Linux**: Casi todas las variables de entorno del sistema (como **\$USER**, **\$SHELL**, **\$PATH**, **\$MAIL**) se escriben **siempre** con **mayúsculas**.



QUESTION 4

Which shell is specified for the htb-student user?

El mismo principio que la pregunta anterior. La shell que está utilizando la terminal se guarda en una **variable de entorno**, por lo tanto si aplicamos lo aprendido en la pregunta anterior respecto de la regla de oro, nuestro comando para obtener la respuesta quedaría de la siguiente manera

`echo $SHELL`

```
htb-student@nixfund:~$ echo $MAIL
/var/mail/htb-student
htb-student@nixfund:~$ echo $SHELL
/bin/bash
```

Question 4

XP +20 ^

Which shell is specified for the htb-student user?

/bin/bash



QUESTION 5

Which kernel release is installed on the system? (Format: 1.22.3)

Acá aplica la lógica de la **pregunta 1**, todo lo que tenga que ver con información básica del sistema se hace con **uname**. Y si no sabemos la opción que necesitamos, ejecutamos primero **man uname** y si revisamos, veremos que la opción que necesitamos es **-r**, por lo tanto con **uname -r** obtendríamos la respuesta.

```
htb-student@nixfund:~$ uname -r  
4.15.0-123-generic
```

Nos piden el formato así que lo ingresamos

Question 5 XP +20 ^

Which kernel release is installed on the system? (Format: 1.22.3)

4.15.0 ❌ ✅



QUESTION 6

What is the name of the network interface that MTU is set to 1500?

Esta es fácil, cuando queremos saber información relacionada a **interfaces de red** es con el comando **ifconfig** y en este caso para responder, es cosa de fijarse al final de la primera línea que nos indica que el **mtu (unidad máxima de transmisión)** está en 1500, por lo tanto la interfaz en este caso sería la **ens192**.

```
htb-student@nixfund:~$ ifconfig
ens192: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.129.44.30 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 dead:beef::a0de:adff:fe7a:5907 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::a0de:adff:fe7a:5907 prefixlen 64 scopeid 0x20<link>
    ether a2:de:ad:7a:59:07 txqueuelen 1000 (Ethernet)
    RX packets 6469 bytes 509532 (509.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1815 bytes 195312 (195.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 3711 bytes 292083 (292.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3711 bytes 292083 (292.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Question 6

 +1 +20

What is the name of the network interface that MTU is set to 1500?



Sección 3: Workflow

Navigation

QUESTION 1

What is the name of the hidden "history" file in the htb-user's home directory?

Antes de responder verificaremos que estamos en la ruta correcta con **pwd**

```
htb-student@nixfund:~$ pwd
/home/htb-student
htb-student@nixfund:~$
```

Para responder a esta pregunta haremos uso del comando **ls**. Este comando por sí solo lo que hace es *"listar"* los archivos de un directorio. Si lo lanzamos nos daremos cuenta que en este caso no nos devuelve nada.

```
htb-student@nixfund:~$ ls
htb-student@nixfund:~$ |
```

Esto ocurre porque el comando **ls** por defecto **no lista los archivos ocultos**, por este motivo, para que los muestre tendremos que agregar opciones al comando. En este caso ejecutamos **ls -a** (a por all) y con eso nos mostrará todos los archivos incluyendo los ocultos.

```
htb-student@nixfund:~$ ls -a
.  ..  .bash_history  .bash_logout  .bashrc  .cache  .gnupg  .profile
htb-student@nixfund:~$ |
```

Si bien con esto ya podemos ver que nos apareció el archivo que buscamos y que responde a *"history"* según la pregunta, ordenaremos un poco la salida.

Si agregamos **l** a las opciones, veremos como la información ahora es más detallada; ejecutamos **ls -la**

```
htb-student@nixfund:~$ ls -a
.  ..  .bash_history  .bash_logout  .bashrc  .cache  .gnupg  .profile
htb-student@nixfund:~$ ls -la
total 32
drwxr-xr-x  4 htb-student htb-student 4096 Aug  3  2021 .
drwxr-xr-x  5 root         root         4096 Aug  3  2021 ..
-rw-r--r--  1 htb-student htb-student   5 Sep 23  2020 .bash_history
-rw-r--r--  1 htb-student htb-student  220 Apr  4  2018 .bash_logout
-rw-r--r--  1 htb-student htb-student 3771 Apr  4  2018 .bashrc
drwx-----  2 htb-student htb-student 4096 Aug  3  2021 .cache
drwx-----  3 htb-student htb-student 4096 Aug  3  2021 .gnupg
-rw-r--r--  1 htb-student htb-student  807 Apr  4  2018 .profile
htb-student@nixfund:~$ |
```

Podemos ver que la salida es diferente, nos entrega información como si se trata de directorios o archivos, los permisos, los enlaces internos al archivo o directorio, el propietario del archivo, el grupo de usuarios al que pertenece, el tamaño (bytes), fecha de modificación y el nombre del archivo.

En este caso la lista es corta y el archivo es fácil de identificar, pero si quisiéramos agregar un filtro que nos liste solo los elementos que tengan **“history”** en su nombre como lo solicita la pregunta, podríamos aplicar el comando **grep** que es básicamente **filtrar**.

En este punto se inserta otro carácter especial de Linux, el **“pipe”** (y el *porqué del nombre de nuestro protagonista*) que es el símbolo **|**, el que se obtiene con la tecla a la izquierda del 1 en el teclado. Este símbolo sirve para agregar una solicitud a la salida, por ejemplo en este caso, **ls -la** obtiene una salida, con **|** puedo pedir otro comando sobre esa salida, como por ejemplo **grep** para aplicar un filtro sobre la salida de la primera parte. Es más fácil de entender haciéndolo.

Pipe y la puerta del reino de Linux | Sección 3: Workflow

```
ls -la | grep history
```

esto lo que hará internamente será ejecutar el mismo comando `ls -la` y en vez de mostrarnos esa salida, aplicará un filtro y buscará aquellos elementos que contengan `history` en su nombre.

```
htb-student@nixfund:~$ ls -la | grep history
-rw----- 1 htb-student htb-student  5 Sep 23  2020 .bash_history
htb-student@nixfund:~$
```

Question 1

XP +20 ^

What is the name of the hidden "history" file in the htb-user's home directory?

SSH to **10.129.44.70 (ACADEMY-NIXFUND)**, with user "**htb-student**" and password "**HTB_@cademy_stdnt!**"

.bash_history



QUESTION 2

What is the index number of the "sudoers" file in the "/etc" directory?

Para responder a esto podemos hacer el camino largo; lo recomiendo para practicar como moverse por el sistema; o el camino corto.

Vamos primero con el camino largo.

Lo primero que haremos será saber dónde estamos con **pwd**

```
htb-student@nixfund:~$ pwd
/home/htb-student
htb-student@nixfund:~$
```

Si lanzamos un **ls -la** podemos ver que no tenemos "ese directorio" **/etc** en el que estaría ese archivo llamado **sudoers**.

```
htb-student@nixfund:~$ ls -la
total 32
drwxr-xr-x 4 htb-student htb-student 4096 Aug  3  2021 .
drwxr-xr-x 5 root         root         4096 Aug  3  2021 ..
-rw----- 1 htb-student htb-student   5 Sep 23  2020 .bash_history
-rw-r--r-- 1 htb-student htb-student  220 Apr  4  2018 .bash_logout
-rw-r--r-- 1 htb-student htb-student 3771 Apr  4  2018 .bashrc
drwx----- 2 htb-student htb-student 4096 Aug  3  2021 .cache
drwx----- 3 htb-student htb-student 4096 Aug  3  2021 .gnupg
-rw-r--r-- 1 htb-student htb-student  807 Apr  4  2018 .profile
htb-student@nixfund:~$ |
```

Por lo que le vamos a preguntar al sistema donde podría estar ese archivo **sudoers** y con eso ver si me dice dónde está el directorio **etc** si es que existe. Utilizaremos **locate** y el nombre del archivo.

```
htb-student@nixfund:~$ locate sudoers
/etc/sudoers
/etc/sudoers.d
/etc/sudoers.d/99-snapd.conf
/etc/sudoers.d/README
/snap/core/10126/etc/sudoers
/snap/core/10126/etc/sudoers.d
/snap/core/10126/etc/sudoers.d/README
/snap/core/10126/usr/lib/sudo/sudoers.la
/snap/core/10126/usr/lib/sudo/sudoers.so
/snap/core/10185/etc/sudoers
/snap/core/10185/etc/sudoers.d
/snap/core/10185/etc/sudoers.d/README
/snap/core/10185/usr/lib/sudo/sudoers.la
/snap/core/10185/usr/lib/sudo/sudoers.so
/snap/core18/1885/etc/sudoers
/snap/core18/1885/etc/sudoers.d
/snap/core18/1885/etc/sudoers.d/README
```

Como se puede ver, **locate** busca y encuentra varias coincidencias al nombre **"sudoers"** y precisamente las primeras hacen mención a ese directorio **/etc**. Al no tener nada antes de **/etc**, puedo asumir que es un **directorio que se encuentra directamente bajo la raíz**, por lo que nos moveremos hacia allá con el comando **cd**.

Si eres nuevo en Linux te recomiendo que hagas este camino, porque por ejemplo con **cd ..** puedo ir un directorio arriba cada vez como muestra la imagen hasta llegar al directorio raíz **/**

```
htb-student@nixfund:~$ pwd
/home/htb-student
htb-student@nixfund:~$ cd ..
htb-student@nixfund:/home$ pwd
/home
htb-student@nixfund:/home$ cd ..
htb-student@nixfund:/$ pwd
/
htb-student@nixfund:/$ |
```

Bajo esa misma lógica si quiero subir más niveles puedo usar **cd ../../..** y así. Por ejemplo para ir arriba 2 niveles de una sola vez utilizo **cd ../../** y

Pipe y la puerta del reino de Linux | Sección 3: Workflow

como muestra la imagen el símbolo ha cambiado de `~` a `/`, indicando con esto que estamos en el directorio raíz.

```
htb-student@nixfund:~$ pwd
/home/htb-student
htb-student@nixfund:~$ cd ../../
htb-student@nixfund:/$ |
```

Tip. Si queremos volver a la carpeta anterior solo ingresamos `cd -`, considerar que si lo haces más de una vez, te llevará atrás y adelante.

Otra forma más directa si quiero ir al directorio raíz es simplemente `cd /`, esto nos llevará al directorio raíz desde cualquier carpeta en la que nos encontremos.

```
htb-student@nixfund:~$ pwd
/home/htb-student
htb-student@nixfund:~$ cd /
htb-student@nixfund:/$ |
```

Ok, ahora avancemos a lo que nos convoca, queremos ir al directorio `/etc`, si hacemos un `ls` en el directorio raíz veremos que aparece el directorio que estamos buscando.

```
htb-student@nixfund:/$ ls
bin  cdrom  etc  initrd.img  lib  lost+found  mnt  proc  run  snap  sys  usr  vmlinuz
boot dev  home  initrd.img.old  lib64  media  opt  root  sbin  srv  tmp  var  vmlinuz.old
htb-student@nixfund:/$ |
```

Con `cd etc` ingresamos al directorio

```
htb-student@nixfund:/$ cd etc
htb-student@nixfund:/etc$ pwd
/etc
htb-student@nixfund:/etc$ |
```

Y una vez dentro del directorio vamos a llamar una nueva opción para `ls`.

La opción será `-i`, por *inode number*. Este número corresponde al identificador único que el sistema le asigna a cada archivo o directorio en

Pipe y la puerta del reino de Linux | Sección 3: Workflow

el disco duro. Probemos con `ls -i sudoers` (nombre del archivo para el que nos están solicitando el número de índice en la pregunta).

```
htb-student@nixfund:/etc$ ls -i sudoers
147627 sudoers
htb-student@nixfund:/etc$
```

Y como se puede ver tenemos la respuesta. Este sería el camino largo.

Probemos ahora el camino corto.

Antes de hacerlo quiero mostrar que con el mismo comando `cd` podemos saltar directamente de un directorio a otro como muestra la imagen. Al final del paso anterior yo estaba en el directorio `/etc`, ingresando la línea de comando `cd /home/htb-student/` pude saltar directo a la carpeta en la que estaba al inicio.

```
htb-student@nixfund:/etc$ cd /home/htb-student/
htb-student@nixfund:~$ pwd
/home/htb-student
htb-student@nixfund:~$ |
```



Pipe y la puerta del reino de Linux | Sección 3: Workflow

Ahora si vamos con el método corto. Si entendimos la lógica de todo lo anterior esto es casi evidente.

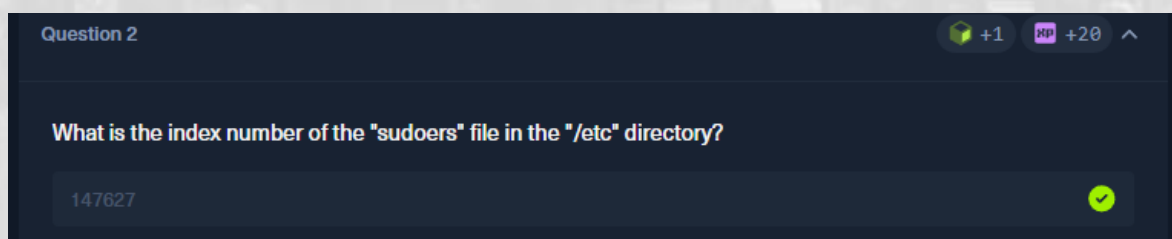
Haremos de cuenta que no se si hay un archivo **sudoers** y si existe en el directorio **etc** , así que hare el primer paso con locate como lo hice antes.

```
htb-student@nixfund:~$ locate sudoers
/etc/sudoers
/etc/sudoers.d
/etc/sudoers.d/99-snapd.conf
/etc/sudoers.d/README
/snap/core/10126/etc/sudoers
/snap/core/10126/etc/sudoers.d
/snap/core/10126/etc/sudoers.d/README
/snap/core/10126/usr/lib/sudo/sudoers.la
/snap/core/10126/usr/lib/sudo/sudoers.so
/snap/core/10185/etc/sudoers
/snap/core/10185/etc/sudoers.d
/snap/core/10185/etc/sudoers.d/README
/snap/core/10185/usr/lib/sudo/sudoers.la
/snap/core/10185/usr/lib/sudo/sudoers.so
/snap/core18/1885/etc/sudoers
/snap/core18/1885/etc/sudoers.d
/snap/core18/1885/etc/sudoers.d/README
```

Ahora que sé que el archivo **sudoers** está en el directorio **etc** simplemente lanzamos el **ls -i** (para solicitar el índice) a la ruta del archivo.

```
htb-student@nixfund:~$ ls -i /etc/sudoers
147627 /etc/sudoers
htb-student@nixfund:~$ |
```

Y eso es todo.



Working with Files and Directories

QUESTION 1

What is the name of the last modified file in the `"/var/backups"` directory?

Para responder esta pregunta utilizaremos la lógica del método “corto” que hicimos al final de la última pregunta de la sección anterior. Pero antes un poco de análisis.

Nos preguntan por un archivo en un directorio, ósea que deberíamos partir por ver una *“lista”* de los archivos disponibles en el directorio. Eso ya nos da la primera pista, el comando iría por el lado de `ls`.

Sabemos que con `ls -la` obtenemos una lista de archivos que incluye los archivos ocultos. Por lo que podemos intuir que este comando puede constituir la base de nuestra búsqueda. Ahora como podemos saber cuál es el último archivo modificado... si tan solo hubiera una forma de *“ordenar por fecha”* como cuando revisamos una carpeta en Windows. Quizás la hay y como no la conozco le preguntare al comando por posibles opciones.

`man ls`

```
LS(1)                                User Commands                                LS(1)
NAME
  ls - list directory contents
SYNOPSIS
  ls [OPTION]... [FILE]...
DESCRIPTION
  List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor
  --sort is specified.
  Mandatory arguments to long options are mandatory for short options too.
  -a, --all
      do not ignore entries starting with .
  -A, --almost-all
      do not list implied . and ..
  --author
      with -l, print the author of each file
  -b, --escape
      print C-style escapes for nongraphic characters
  --block-size=SIZE
      scale sizes by SIZE before printing them; e.g., '--block-size=M' prints sizes in units of 1,048,576 bytes; see SIZE for-
      mat below
```

Pipe y la puerta del reino de Linux | Sección 3: Workflow

Como se ve en la imagen anterior obtenemos en pantalla las posibles opciones, ahora buscaremos si existe algo para la fecha de modificación. Si bajamos en la salida del comando `man ls` nos encontraremos con que la opción `-t` es la que estoy buscando como lo muestra la imagen siguiente.

```
With -t, show times using style STYLE. Full-ISO, long-ISO, ISO,
'date'; if FORMAT is FORMAT1<newline>FORMAT2, then FORMAT1 applies to n
STYLE is prefixed with 'posix-', STYLE takes effect only outside the PO

-t      sort by modification time, newest first

-T, --tabsize=COLS
        assume tab stops at each COLS instead of 8

-u      with -lt: sort by, and show, access time; with -l: show access time
        newest first
```

Por lo que probaremos a incluir esa opción, por lo tanto nuestro comando quedaría así: `ls -lat` y la ruta en la que queremos buscar dicho archivo

`ls -lat /var/backups`

y como muestra la imagen siguiente, el segundo archivo, `apt.extended_states.0` sería el que estamos buscando.

```
htb-student@nixfund:~$ ls -lat /var/backups
total 2168
drwxr-xr-x  2 root root    4096 Aug  3  2021 .
-rw-r--r--  1 root root   41872 Nov 12  2020 apt.extended_states.0
-rw-r--r--  1 root root   4437  Nov 12  2020 apt.extended_states.1.gz
-rw-r--r--  1 root root  742750 Nov 11  2020 dpkg.status.0
-rw-r--r--  1 root root  206270 Nov 11  2020 dpkg.status.1.gz
```

Si ponemos atención a la salida, nos damos cuenta que los archivos están en un orden que coloca a los más nuevos primero. Esto no tiene mucha importancia cuando se trata de una lista no muy larga, pero en la práctica habrán salidas con listas tan largas que tendremos que hacer mucho scroll en pantalla para volver hacia arriba lo que puede resultar un poco molesto.

Si tan solo hubiese una forma de invertir la lista.

Y ciertamente que la hay, si ponemos atención a la salida de `man ls`, podemos ver que hay una opción para invertir la salida, `-r`

```
--quoting-style=WORD
    use quoting style WORD for entry names: literal
    escape

-r, --reverse
    reverse order while sorting

-R, --recursive
    list subdirectories recursively
```

Por lo que nuestro comando ahora quedaría como `ls -latr`

```
-rw-r--r-- 1 root root 206270 Nov  5  2020 dpkg.status.2.gz
-rw-r--r-- 1 root root 206270 Nov 11  2020 dpkg.status.1.gz
-rw-r--r-- 1 root root 742750 Nov 11  2020 dpkg.status.0
-rw-r--r-- 1 root root  4437 Nov 12  2020 apt.extended_states.1.gz
-rw-r--r-- 1 root root  41872 Nov 12  2020 apt.extended_states.0
drwxr-xr-x 2 root root  4096 Aug  3  2021 .
htb-student@nixfund:~$ |
```

Y podemos ver que ahora la salida aparece invertida, los más nuevos abajo.

Aclaración: Por si alguien piensa que ese `.` que aparece al final es posiblemente la respuesta, solo basta mirar a la izquierda, esa última línea comienza con la letra `d` y eso indica que es un **directorio** y nos están preguntando por un archivo, por eso la respuesta es:

Question 1

hp +20 ^

What is the name of the last modified file in the "/var/backups" directory?

SSH to 10.129.44.117 (ACADEMY-NIXFUND), with user "htb-student" and password "HTB_@cademy_stdnt!"

apt.extended_states.0 ✓

QUESTION 2

What is the inode number of the "shadow.bak" file in the "/var/backups" directory?

Ya sabemos que el **inode** lo obtenemos con **ls -i**, por lo que solo sería hacer un recuento de cosas que ya hemos revisado para poder responder.

```
ls -i /ruta_archivo
```

```
htb-student@nixfund:~$ ls -i /var/backups/shadow.bak
265293 /var/backups/shadow.bak
htb-student@nixfund:~$ |
```

Question 2

+1

+20

^

What is the inode number of the "shadow.bak" file in the "/var/backups" directory?



Find Files and Directories

QUESTION 1

What is the name of the config file that has been created after 2020-03-03 and is smaller than 28k but larger than 25k?

Para poder responder lo primero que debemos tener claro respecto del archivo sobre el que nos piden el nombre es que **no sabemos**:

- su ubicación; por lo que la búsqueda deberá ser en todo el sistema.
- el nombre del archivo; por lo que puede ser cualquier cosa.

Lo que **sí sabemos** es:

- se trata de un archivo de configuración por lo que probablemente tenga algo en el nombre como **.conf**
- que fue creado después del 3 de marzo de 2020
- que su tamaño esta entre 25k y 28k.

Para buscar algo habíamos utilizado previamente el comando **locate**, pero eso funcionaba cuando teníamos el nombre como por ejemplo **shadow.bak**

```
htb-student@nixfund:~$ locate shadow.bak
/var/backups/gshadow.bak
/var/backups/shadow.bak
htb-student@nixfund:~$
```

Por este motivo utilizaremos el comando **find** con opciones. Para ver las opciones, adivina que usaremos... tal cual, **man find** .

Al comenzar a ver las opciones vemos algunas que nos pueden servir para armar nuestro comando para encontrar ese archivo específico.

Antes de comenzar a agregar opciones a nuestro comando `find` es entender una parte básica del mismo. El comando funciona llamándolo y agregando una ruta en la que buscar más lo que estamos buscando (tipo* y nombre); por ejemplo, si tomamos el último archivo de la pregunta anterior *“shadow.bak”* la forma de buscarlo sería:

1. Ingresamos el comando
`find`
2. Indicamos la ruta
`/var/backups`
3. Indicamos el tipo (no obligatorio... es una opción)
`-type f`
4. Indicamos el nombre de lo que estamos buscando
`-name “shadow.bak”`

```
htb-student@nixfund:~$ find /var/backups -type f -name "shadow.bak"
/var/backups/shadow.bak
htb-student@nixfund:~$ find /var/backups -name "shadow.bak"
/var/backups/shadow.bak
htb-student@nixfund:~$ |
```

Aunque en este caso es un poco *“ridícula”* la búsqueda porque ya sé dónde está y como se llama, pero creo que sirve para explicar el orden. Notar además que en este caso el `-type f` no era necesario, hay otros casos que sí y para eso sirve revisar `man find`

Ahora si aplicamos esta lógica a nuestro ejercicio, como señalamos al inicio, hay cosas que sabemos y cosas que no sabemos.

Entre las que no sabemos está la *ubicación* por lo que la búsqueda tendrá que ser en **todo el sistema**. Para buscar en todo el sistema tendríamos que aplicar lo que aprendimos antes, el **directorio raíz**, ósea `/`, por lo tanto esa primera parte quedaría como `find /`


```
htb-student@nixfund:~$ find /
```

Para el tipo (**type -f**) sigue siendo archivo (**f**), no existe un **type -conf** o algo así; insisto que revisar el comando **man find** sirve para despejar este tipo de dudas.

```
-type c
File is of type c:

b      block (buffered) special
c      character (unbuffered) special
d      directory
p      named pipe (FIFO)
f      regular file
l      symbolic link; this is never true if the -L option or the -follow option is in effect, unless the
      symbolic link is broken.  If you want to search for symbolic links when -L is in effect, use
      -xtype.
s      socket
D      door (Solaris)
```

Ahora nos faltaría el **nombre** de lo que vamos a buscar que es otro item que tampoco conocemos, pero si sabemos que es un archivo de configuración como dice la pregunta (config file) y esos archivos en Linux tienen **.conf** en el nombre. Como no sabemos el nombre específico antes del **.conf** entonces debemos buscar todos los archivos con **.conf** ; esto se lleva a cabo con el símbolo ***** , por lo que en nuestra línea de comando quedaría de la siguiente forma: **-iname "*.conf"** (entre comillas).

```
-name pattern
Base of file name (the path with the leading directories removed) matches shell pattern pattern. Because
the leading directories are removed, the file names considered for a match with -name will never include a
slash, so '-name a/b' will never match anything (you probably need to use -path instead). A warning is
issued if you try to do this, unless the environment variable POSIXLY_CORRECT is set. The metacharacters
('*', '?', and '[') match a '.' at the start of the base name (this is a change in findutils-4.2.2; see
section STANDARDS CONFORMANCE below). To ignore a directory and the files under it, use -prune; see an
example in the description of -path. Braces are not recognised as being special, despite the fact that
some shells including Bash imbue braces with a special meaning in shell patterns. The filename matching
is performed with the use of the fnmatch(3) library function. Don't forget to enclose the pattern in
quotes in order to protect it from expansion by the shell.
```

Si juntamos todo esto, nuestro comando se empieza a armar de la siguiente manera, **find / -type f -iname "*.conf"**

```
htb-student@nixfund:~$ find / -type f -iname "*.conf"
```

Importante. Con `-i` en `-iname` lo que logramos es que el programa ignore entre mayúsculas y minúsculas.

Si ejecutamos el comando así como esta nos enviará una larga lista por lo que aún nos falta agregar otras opciones para afinar la búsqueda.

```
htb-student@nixfund:~$ find / -type f -iname "*.conf"
find: '/sys/kernel/debug': Permission denied
find: '/sys/fs/pstore': Permission denied
find: '/sys/fs/fuse/connections/50': Permission denied
find: '/sys/fs/fuse/connections/45': Permission denied
find: '/lost+found': Permission denied
find: '/home/mrb3n/.gnupg': Permission denied
find: '/home/mrb3n/.cache': Permission denied
find: '/home/mrb3n/.ssh': Permission denied
find: '/home/mrb3n/.local/share': Permission denied
find: '/home/cry0llt3/.gnupg': Permission denied
find: '/home/cry0llt3/.cache': Permission denied
find: '/home/cry0llt3/.ssh': Permission denied
find: '/home/cry0llt3/.local/share': Permission denied
find: '/var/spool/rsyslog': Permission denied
find: '/var/spool/postfix/incoming': Permission denied
find: '/var/spool/postfix/maildrop': Permission denied
find: '/var/spool/postfix/flush': Permission denied
find: '/var/spool/postfix/bounce': Permission denied
find: '/var/spool/postfix/defer': Permission denied
find: '/var/spool/postfix/public': Permission denied
find: '/var/spool/postfix/corrupt': Permission denied
/var/spool/postfix/etc/nsswitch.conf
/var/spool/postfix/etc/resolv.conf
/var/spool/postfix/etc/host.conf
find: '/var/spool/postfix/trace': Permission denied
find: '/var/spool/postfix/deferred': Permission denied
```

Dentro de información que si tenemos para realizar la búsqueda esta la **fecha de creación posterior al 3 de marzo de 2020.**

Nuestro comando `man find` nos entregó la opción para esto.

```
-newerXY reference
Succeeds if timestamp X of the file being considered is newer than timestamp Y of the file reference.
The letters X and Y can be any of the following letters:

a The access time of the file reference
B The birth time of the file reference
c The inode status change time of reference
m The modification time of the file reference
t reference is interpreted directly as a time

Some combinations are invalid; for example, it is invalid for X to be t. Some combinations are not implemented on all systems; for example B is not supported on all systems. If an invalid or unsupported combination of XY is specified, a fatal error results. Time specifications are interpreted as for the argument to the -d option of GNU date. If you try to use the birth time of a reference file, and the birth time cannot be determined, a fatal error message results. If you specify a test which refers to the birth time of files being examined, this test will fail for any files where the birth time is unknown.
```

Pipe y la puerta del reino de Linux | Sección 3: Workflow

Y nuestra línea de comando ahora agrega la fecha de creación quedando:

```
htb-student@nixfund:~$ find / -type f -iname "*.conf" -newerct 2020-03-03
find: '/sys/kernel/debug': Permission denied
find: '/sys/fs/pstore': Permission denied
find: '/sys/fs/fuse/connections/50': Permission denied
```

La salida es gigante

```
htb-student@nixfund:~$ find / -type f -name "*.conf" -newerct 2020-03-03
find: '/sys/kernel/debug': Permission denied
find: '/sys/fs/pstore': Permission denied
find: '/sys/fs/fuse/connections/50': Permission denied
find: '/sys/fs/fuse/connections/45': Permission denied
find: '/lost+found': Permission denied
find: '/home/mrb3n/.gnupg': Permission denied
find: '/home/mrb3n/.cache': Permission denied
find: '/home/mrb3n/.ssh': Permission denied
find: '/home/mrb3n/.local/share': Permission denied
find: '/home/cry@lit3/.gnupg': Permission denied
find: '/home/cry@lit3/.cache': Permission denied
find: '/home/cry@lit3/.ssh': Permission denied
find: '/var/spool/postfix/defer': Permission denied
find: '/var/spool/postfix/incoming': Permission denied
find: '/var/spool/postfix/maildrop': Permission denied
find: '/var/spool/postfix/flush': Permission denied
find: '/var/spool/postfix/bounce': Permission denied
find: '/var/spool/postfix/defer': Permission denied
find: '/var/spool/postfix/public': Permission denied
find: '/var/spool/postfix/corrupt': Permission denied
find: '/var/spool/postfix/trace': Permission denied
find: '/var/spool/postfix/deferred': Permission denied
find: '/var/spool/postfix/hold': Permission denied
find: '/var/spool/postfix/active': Permission denied
find: '/var/spool/postfix/private': Permission denied
find: '/var/spool/postfix/saved': Permission denied
find: '/var/spool/cron/at.spool': Permission denied
find: '/var/spool/cron/crontabs': Permission denied
find: '/var/cache/apt/archives/partial': Permission denied
find: '/var/log/unattended-upgrades': Permission denied
find: '/var/log/mysql': Permission denied
find: '/var/log/samba': Permission denied
find: '/var/log/apache2': Permission denied
find: '/var/lib/updates-notifier/package-data-downloads/partial': Permission denied
find: '/var/lib/php/sessions': Permission denied
find: '/var/lib/mysql': Permission denied
find: '/var/lib/mysql-files': Permission denied
find: '/var/lib/samba/usershares': Permission denied
find: '/var/lib/samba/private/msg.sock': Permission denied
find: '/var/lib/snapd/cookie': Permission denied
find: '/var/lib/snapd/cache': Permission denied
find: '/var/lib/snapd/void': Permission denied
/var/lib/dpkg/info/mdadm.config
/var/lib/dpkg/info/openssh-server.config
/var/lib/dpkg/info/landscape-common.config
/var/lib/dpkg/info/unattended-upgrades.config
/var/lib/dpkg/info/trdata.config
/var/lib/dpkg/info/ca-certificates.config
/var/lib/dpkg/info/irqbalance.config
/var/lib/dpkg/info/debconf.config
/var/lib/dpkg/info/console-setup.config
/var/lib/dpkg/info/ufw.config
/var/lib/dpkg/info/popularity-contest.config
/var/lib/dpkg/info/adduser.config
/var/lib/dpkg/info/grub-pc.config
/var/lib/dpkg/info/locales.config
/var/lib/dpkg/info/apparmor.config
/var/lib/dpkg/info/mysql-server-5.7.config
/var/lib/dpkg/info/man-db.config
/var/lib/dpkg/info/byobu.config
/var/lib/dpkg/info/dash.config
/var/lib/dpkg/info/postfix.config
/var/lib/dpkg/info/cloud-init.config
/var/lib/dpkg/info/samba-common.config
/var/lib/dpkg/info/newboard-configuration.config
find: '/var/lib/mysql-keyring': Permission denied
find: '/var/lib/apt/lists/partial': Permission denied
find: '/var/lib/private': Permission denied
find: '/var/lib/polkit-1': Permission denied
find: '/var/tmp/systemd-private-68dbf4991dca4fala7d1cfff3fbc-Systemd-resolved.service-the00': Permission denied
find: '/var/tmp/systemd-private-68dbf4991dca4fala7d1cfff3fbc-Systemd-resolved.service-the00': Permission denied
find: '/var/tmp/systemd-private-68dbf4991dca4fala7d1cfff3fbc-Systemd-timesyncd.service-K8um2r': Permission denied
find: '/var/tmp/systemd-private-68dbf4991dca4fala7d1cfff3fbc-Systemd-timesyncd.service-K8um2r': Permission denied
find: '/etc/dovecot/private': Permission denied
/etc/manpath.config
find: '/etc/ssl/private': Permission denied
find: '/etc/pki/3rdparty/updates': Permission denied
```

Aquí algunos puedan pensar que podría ser **-newerbt** en vez de la opción **-newerct** y esto nos daría un resultado incorrecto para esta búsqueda porque estamos buscando una fecha **“posterior a”** y no **“igual a”**.

Pipe y la puerta del reino de Linux | Sección 3: Workflow

El último dato que conocemos y que debemos agregar es el **tamaño** que está en un **rango entre 25k y 28k**.

```
-size n[cwbkMG]
File uses n units of space, rounding up. The following suffixes can be used:

'b'   for 512-byte blocks (this is the default if no suffix is used)
'c'   for bytes
'w'   for two-byte words
'k'   for Kibibytes (KiB, units of 1024 bytes)
'M'   for Mebibytes (MiB, units of 1024 * 1024 = 1048576 bytes)
'G'   for Gibibytes (GiB, units of 1024 * 1024 * 1024 = 1073741824 bytes)

The size does not count indirect blocks, but it does count blocks in sparse files that are not actually allocated. Bear in mind that the '%k' and '%b' format specifiers of -printf handle sparse files differently. The 'b' suffix always denotes 512-byte blocks and never 1024-byte blocks, which is different to the behaviour of -ls.
```

Para esto agregaremos con `-size` como nos indica `man find`

```
htb-student@nixfund:~$ find / -type f -iname "*.conf" -newerct 2020-03-03 -size +25k -size -28k
find: '/sys/kernel/debug': Permission denied
find: '/sys/fs/pstore': Permission denied
find: '/sys/fs/fuse/connections/50': Permission denied
find: '/sys/fs/fuse/connections/45': Permission denied
find: '/lost+found': Permission denied
find: '/home/mrb3n/.gnupg': Permission denied
find: '/home/mrb3n/.cache': Permission denied
```

Pero la salida sigue siendo gigante... *¿entonces que estamos haciendo mal?* Las opciones están bien por lo que sabemos.

[illegible]

Si ponemos atención a la salida nos daremos cuenta que se repite mucho:

“Permission denied”

```
htb-student@nixfund:~$ find / -type f -name "*.config" -newerct 2020-03-03 -size +25k -size -28k
find: '/sys/kernel/debug': Permission denied
find: '/sys/fs/pstore': Permission denied
find: '/sys/fs/fuse/connections/50': Permission denied
find: '/sys/fs/fuse/connections/45': Permission denied
find: '/lost+found': Permission denied
find: '/home/mrb3n/.gnupg': Permission denied
```

¿Y si hubiese una forma de pedirle a la salida que no nos muestre todos esos mensajes de error y que deje solo la información que nos sirve?

Afortunadamente en Linux tenemos esa opción y se llama

2>/dev/null

Para la respuesta técnica se puede buscar en internet, que de seguro habrá miles de respuestas mejor que la que yo pueda dar, pero en mis palabras sería tan simple como que con este agregado, le pedimos al sistema que los mensajes de error del comando sean enviados o descartados a un ***“agujero negro”*** y por lo tanto no me los muestre. Por lo tanto nuestra línea de comando quedaría así.

find / -type f -iname "*.conf" -newerct 2020-03-03 -size +25k -size -28k 2>/dev/null

y voila!

```
htb-student@nixfund:~$ find / -type f -iname "*.conf" -newerct 2020-03-03 -size +25k -size -28k 2>/dev/null
/usr/share/dirc.d/00-mesa-defaults.conf
htb-student@nixfund:~$
```

Question 1 +1 +20 ^

What is the name of the config file that has been created after 2020-03-03 and is smaller than 28k but larger than 25k?

SSH to **10.129.44.117 (ACADEMY-NIXFUND)**, with user "**htb-student**" and password "**HTB_@cademy_stdnt!**"

00-mesa-defaults.conf ✓

Un buen recurso adicional:

https://www.linuxtotal.com.mx/index.php?cont=info_admon_022



QUESTION 2

How many files exist on the system that have the ".bak" extension?

Mismo razonamiento. En esta ocasión nos preguntan por una cantidad, por lo tanto la salida será un número. También dice *“on the system”*, ósea que buscaremos en todo el sistema `/`. Y por último sabemos que tienen extensión *.bak*

Veamos si `find` nos ayuda.

Podemos ver en la siguiente salida que está funcionando pero que nuevamente los mensajes de error se toman la salida.

```
htb-student@nixfund:~$ find / -type f -name "*.bak"
find: '/sys/kernel/debug': Permission denied
find: '/sys/fs/pstore': Permission denied
find: '/sys/fs/fuse/connections/50': Permission denied
find: '/sys/fs/fuse/connections/45': Permission denied
find: '/lost+found': Permission denied
find: '/home/mrb3n/.gnupg': Permission denied
```

Por lo que refinaremos y agregaremos el ya explicado `2>/dev/null`

```
htb-student@nixfund:~$ find / -type f -name "*.bak" 2>/dev/null
/var/backups/shadow.bak
/var/backups/gshadow.bak
/var/backups/group.bak
/var/backups/passwd.bak
htb-student@nixfund:~$ |
```

Y como se puede apreciar en la imagen de arriba, vemos que tenemos 4 archivos y por lo tanto la respuesta sería 4. Es fácil contarlas porque son solo 4 líneas, pero *¿Qué sucedería si fuesen 437 líneas? ¿las contaríamos de a una?* Lo más seguro es que perdamos la cuenta. Entonces debe haber un modo de contar esas líneas de la salida.

Como había explicado previamente, en la **QUESTION 1** de la **Sección 3**, tenemos las “pipes” representadas por el símbolo `|` y que sirven para

Pipe y la puerta del reino de Linux | Sección 3: Workflow

agregar acciones a una salida. En este caso, quiero que sobre la salida actual, aplique un contador.

Una buena forma de contar palabras sería con el comando **grep** que también explique en esa sección. Si a **grep** le agregamos la opción **-c** y el texto que estamos buscando, entonces contará el número de coincidencias. En este caso queda así.

```
find / -type f -name "*.bak" 2>/dev/null | grep -c bak
```

```
htb-student@nixfund:~$ find / -type f -name "*.bak" 2>/dev/null | grep -c bak
4
htb-student@nixfund:~$
```

Si buscamos otra coincidencia el resultado es distinto.

```
htb-student@nixfund:~$ find / -type f -name "*.bak" 2>/dev/null | grep -c shadow
2
```

Por lo tanto la respuesta correcta queda así:

Question 2 +1 MP +20 ^

How many files exist on the system that have the ".bak" extension?

✓

QUESTION 3

Submit the full path of the "xxd" binary.

Si seguimos lo aprendido en las anteriores preguntas de esta sección, podríamos responder a esta pregunta con **find** y sacando un par de conclusiones. Lo único que sabemos es que se trata de un archivo **binario** llamado **xxd**. Así que si realizamos esa búsqueda:

```
find / -type f -iname "xxd" 2>/dev/null
```

```
htb-student@nixfund:~$ find / -type f -iname "xxd" 2>/dev/null
/usr/bin/xxd
/usr/share/bash-completion/completions/xxd
/snap/core/10126/usr/bin/xxd
/snap/core/10126/usr/share/bash-completion/completions/xxd
/snap/core/10185/usr/bin/xxd
/snap/core/10185/usr/share/bash-completion/completions/xxd
/snap/core18/1932/usr/bin/xxd
/snap/core18/1932/usr/share/bash-completion/completions/xxd
/snap/core18/1885/usr/bin/xxd
/snap/core18/1885/usr/share/bash-completion/completions/xxd
```

Podemos ver que la salida muestra varias rutas y tal vez una de esas sea.

Ahora, para responder de modo más eficiente y certero, debemos entender antes una cosa. *¿Qué es un archivo .bin?* Como siempre insisto, buscar más información en Google para obtener respuestas más detalladas, pero así muy en corto y directo, **los archivos .bin son ejecutables**. Así no más. *¿Y esto porque es importante en este contexto?* Porque este tipo de archivos dejan lo que se conoce como **“enlaces simbólicos”** (equivalentes a los accesos directos de Windows) en **\$PATH**. *¿Y que es \$PATH ?* Son **rutas del sistema**, es una carpeta que el sistema se sabe de memoria y es donde estarían esos **“enlaces simbólicos”** que comandos como **ls** o **python** utilizan para ejecutar los archivos **.bin**

```
htb-student@nixfund:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```


Pipe y la puerta del reino de Linux | Sección 3: Workflow

Y aquí es donde aparece un nuevo comando que es el que nos sirve para responder a la pregunta... **which**.

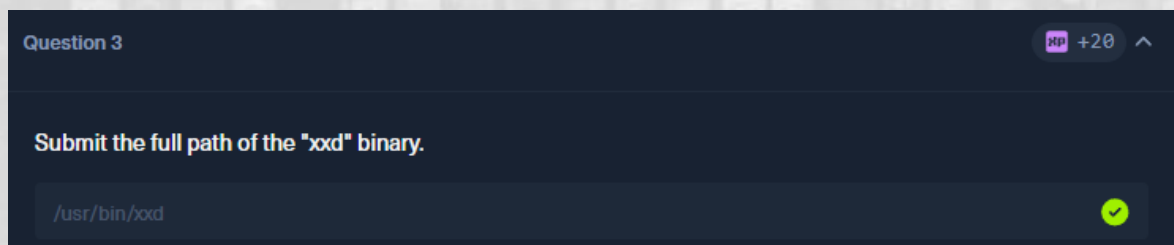
El comando **which** encuentra el acceso directo (**enlace simbólico**) o el programa real, pero devuelve la ruta del acceso directo en sí, no la del archivo final al que apunta. A diferencia del comando **find**, **which** se concentra solo en archivos binarios.

Ejemplo:

Si ejecutamos **which python**, nos dirá **/usr/bin/python** (que es un enlace simbólico), no **/System/Volumes/Data/usr/local/bin/python3.11** (el archivo real). Espero se entienda jaja.

Así que dicho lo dicho, ejecutamos **which xxd**

```
htb-student@nixfund:~$ which xxd
/usr/bin/xxd
htb-student@nixfund:~$ |
```



Comando	¿Qué busca?	¿Dónde busca?	¿Cuándo usarlo? (Aplicación)
which	Solo ejecutables/comandos.	En las carpetas del \$PATH.	Para saber la ruta exacta de un comando (ej. which python).
find	Cualquier archivo/carpeta (en tiempo real).	En el disco duro (en la ruta que le digas).	Para búsquedas complejas (ej. archivos modificados hoy o por tamaño).
locate	Cualquier archivo/carpeta (instantáneo).	En una base de datos indexada previamente.	Para encontrar un archivo rápido por su nombre sin ralentizar el PC. Se debe actualizar la bbdd con sudo updatedb

File Descriptors and Redirections

QUESTION 1

How many files exist on the system that have the ".log" file extension?

Para responder simplemente hacemos un compendio de lo aprendido.

- Preguntan por cantidad por lo que podemos utilizar el **grep -c**
- La extensión indica que se trata de un archivo **.log** y no un **.bin** por lo que podemos descartar **which** e inclinarnos por **find**.
- No dan una ruta específica por lo que la búsqueda será en todo el sistema y por lo tanto utilizaremos **/**.
- No sabemos los nombres, solo las extensiones **.log** por lo que la búsqueda deberá traer todos los archivos con esa extensión y para eso utilizaremos la opción **-name "*.log"**

Si juntamos todo eso, el comando entonces quedaría así.

```
find / -type f -name "*.log" 2>/dev/null | grep -c log
```

```
htb-student@nixfund:~$ find / -type f -name "*.log" 2>/dev/null | grep -c log
32
htb-student@nixfund:~$ |
```

Question 1

+1 XP +20

How many files exist on the system that have the ".log" file extension?

SSH to 10.129.45.78 (ACADEMY-NIXFUND), with user "htb-student" and password "HTB_academy_stdnt!"

32

✓

QUESTION 2

How many total packages are installed on the target system?

Para poder responder esto, primero debemos saber en qué distribución de Linux estamos trabajando. No entraré a explicar todas las distribuciones; hay mucha información en Google, Youtube y los modelos de Inteligencia Artificial que pueden aclarar dudas de forma mucho más eficiente y certera de lo que podría hacerlo yo en esta guía. Solo diré que a grandes rasgos, la imagen muestra las grandes distribuciones de Linux.



Enlace recomendado para ver la línea de tiempo y las distros de Linux:
https://upload.wikimedia.org/wikipedia/commons/1/1b/Linux_Distribution_Timeline.svg

Para saber en qué distribución de Linux estamos, utilizaremos el siguiente comando, **cat**; por si no lo sabes, **cat** es un comando que se utiliza principalmente para leer contenido de un archivo, en este caso el archivo contiene la información de la **release** del sistema operativo.

cat /etc/os-release

```
htb-student@nixfund:~$ cat /etc/os-release
NAME="Ubuntu"
VERSION="18.04.5 LTS (Bionic Beaver)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 18.04.5 LTS"
VERSION_ID="18.04"
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
VERSION_CODENAME=bionic
UBUNTU_CODENAME=bionic
```

Como podemos ver en la imagen en el campo **ID = ubuntu** nos indica que se trata precisamente de **Ubuntu**. Si vemos la imagen de distros de Linux, podemos notar que Ubuntu es una distribución que nace de **Debian**.

¿Por qué esto es importante?

Porque el sistema de gestión de paquetes de Debian son **dpkg** (manual) y **apt** (automático). Esto quiere decir:

Si quiero instalar un programa con **dpkg** y falta alguna dependencia, el programa no se instalará o no funcionará como debe hasta que resuelvan manualmente esas dependencias faltantes, la instalación fallará a mitad de camino o quedará rota (incompleta) porque no sabe ir a buscar lo que le falta, nos toca a nosotros buscar, descargar e instalar cada dependencia a mano.... pero en cambio, si lo hacemos con el gestor de paquetes **apt**, el analiza lo que falta, se conecta a los repositorios de internet, descarga de forma automática todas las dependencias necesarias y las instala en el orden correcto sin que nosotros movamos un dedo.

Pipe y la puerta del reino de Linux | Sección 3: Workflow

Si estuviéramos en un sistema que fuese **Red Hat** o alguna distro que nazca de esa rama los gestores equivalentes serían **rpm** y **dnf**.

Entonces con esta información podemos hacer la búsqueda en nuestro sistema, sabemos que puede ser **dpkg** o **apt**.

Si lo hacemos con **apt**, tenemos la opción de listar con **list** y solicitar que nos muestre solo los instalados con **--installed**. Si armamos ese comando queda **apt list --installed** y lo que nos entrega es una lista de todos los programas instalados:

```
htb-student@nixfund:~$ apt list --installed
Listing... Done
accountsservice/bionic-updates,bionic-security,now 0.6.45-1ubuntu1.3 amd64 [installed,automatic]
acl/bionic,now 2.2.52-3build1 amd64 [installed,automatic]
acpid/bionic,now 1:2.0.28-1ubuntu1 amd64 [installed,automatic]
adduser/bionic,now 3.116ubuntu1 all [installed]
adwaita-icon-theme/bionic,now 3.28.0-1ubuntu1 all [installed,automatic]
amd64-microcode/bionic-updates,bionic-security,now 3.20191021.1+really3.20181128.1~ubuntu0.18.04.1 amd64 [installed]
apache2/bionic-updates,bionic-security,now 2.4.29-1ubuntu4.14 amd64 [installed]
apache2-bin/bionic-updates,bionic-security,now 2.4.29-1ubuntu4.14 amd64 [installed,automatic]
apache2-data/bionic-updates,bionic-security,now 2.4.29-1ubuntu4.14 all [installed,automatic]
apache2-utils/bionic-updates,bionic-security,now 2.4.29-1ubuntu4.14 amd64 [installed,automatic]
apparmor/bionic-updates,bionic-security,now 2.12-4ubuntu5.1 amd64 [installed,automatic]
appport/bionic-updates,now 2.20.9-0ubuntu7.19 all [installed,automatic]
appport-symptoms/bionic,now 0.20 all [installed,automatic]
apt/bionic-updates,bionic-security,now 1.6.12ubuntu0.1 amd64 [installed]
apt-utils/bionic-updates,bionic-security,now 1.6.12ubuntu0.1 amd64 [installed]
at/bionic,now 3.1.20-3.1ubuntu2 amd64 [installed,automatic]
at-spi2-core/bionic,now 2.28.0-1 amd64 [installed,automatic]
attr/bionic,now 1:2.4.47-2build1 amd64 [installed,automatic]
base-files/bionic-updates,now 10.1ubuntu2.10 amd64 [installed,automatic]
base-passwd/bionic,now 3.5.44 amd64 [installed,automatic]
bash/bionic-updates,now 4.4.18-2ubuntu1.2 amd64 [installed]
bash-completion/bionic,now 1:2.8-1ubuntu1 all [installed,automatic]
bc/bionic,now 1.07.1-2 amd64 [installed,automatic]
bcache-tools/bionic-updates,now 1.0.8-2ubuntu0.18.04.1 amd64 [installed,automatic]
bind9-host/bionic-updates,bionic-security,now 1:9.11.3+dfsg-1ubuntu1.13 amd64 [installed,automatic]
bsdmainutils/bionic,now 11.1.2ubuntu1 amd64 [installed,automatic]
bsdutils/bionic-updates,bionic-security,now 1:2.31.1-0.4ubuntu3.7 amd64 [installed,automatic]
btrfs-progs/bionic,now 4.15.1-1build1 amd64 [installed,automatic]
btrfs-tools/bionic,now 4.15.1-1build1 amd64 [installed,automatic]
busybox-initramfs/bionic-updates,bionic-security,now 1:1.27.2-2ubuntu3.3 amd64 [installed,automatic]
busybox-static/bionic-updates,bionic-security,now 1:1.27.2-2ubuntu3.3 amd64 [installed,automatic]
byobu/bionic,now 5.125-0ubuntu1 all [installed,automatic]
bzip2/bionic-updates,bionic-security,now 1.0.6-8.1ubuntu0.2 amd64 [installed]
ca-certificates/bionic-updates,bionic-security,now 20201027ubuntu0.18.04.1 all [installed,automatic]
cloud-guest-utils/bionic,now 0.30-0ubuntu5 all [installed,automatic]
cloud-init/bionic-updates,now 20.3-2-g371b392c-0ubuntu1~18.04.1 all [installed]
cloud-initramfs-copymods/bionic-updates,now 0.40ubuntu1.1 all [installed,automatic]
cloud-initramfs-dyn-netconf/bionic-updates,now 0.40ubuntu1.1 all [installed,automatic]
command-not-found/bionic-updates,now 18.04.5 all [installed,automatic]
command-not-found-data/bionic-updates,now 18.04.5 amd64 [installed,automatic]
```

Pipe y la puerta del reino de Linux | Sección 3: Workflow

Pero a nosotros nos están pidiendo un número que indique la cantidad de paquetes instalados. Por lo tanto solo agregaremos nuestro `grep -c`

`apt list --installed | grep -c installed`

```
htb-student@nixfund:~$ apt list --installed | grep -c installed
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
737
htb-student@nixfund:~$
```

También hay una forma alternativa; como vimos antes, **Debian** gestiona los paquetes con **apt** y con **dpkg** por lo que probamos.

```
htb-student@nixfund:~$ dpkg -l | grep -c installed
2
```

Pero claramente algo no está bien, uno me dice **737** y el otro **2**; de modo que probaremos lo siguiente.

`dpkg -l | grep -c "^ii"`

```
htb-student@nixfund:~$ dpkg -l | grep -c "^ii"
737
```

Y ahora si funcionó... ¿Qué sucedió?

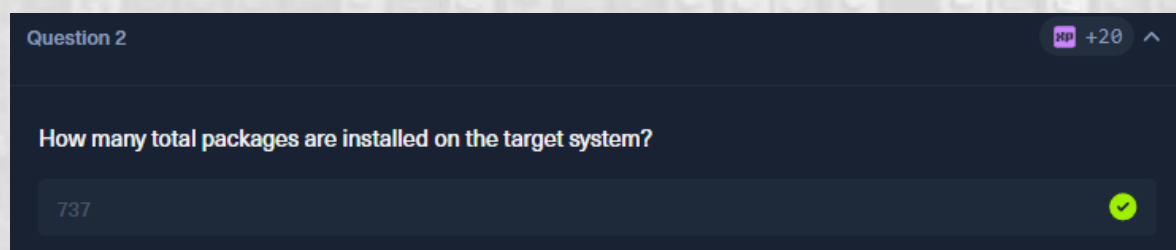
La diferencia está en qué palabra exacta estamos buscando en cada comando:

- `grep -c "^ii"` (Da 737): Es el número real. Busca las líneas que comienzan (^) con **ii**. En **dpkg**, las letras **ii** significan **"paquete instalado correctamente"**.
- `grep -c installed` (Da 2): Solo busca la palabra literal **"installed"**. En la lista de **dpkg**, esa palabra exacta casi no se usa, excepto en un par de líneas de texto del encabezado de ayuda o comentarios (por eso solo encuentra 2).

Pipe y la puerta del reino de Linux | Sección 3: Workflow

En resumen: `^ii` filtra por el código de estado de Linux, mientras que `installed` solo busca texto plano en inglés.

Y por estas razones, **737** es la respuesta correcta.



Filter Contents

Recomiendo hacer y no solo leer los ejemplos y la práctica de la parte teórica, tocan varios comandos que un usuario de Linux debe manejar.

1. A line with the username cry0l1t3.

```
cat /etc/passwd | grep cry0l1t3 | cut -d":" -f1
```

```
htb-student@nixfund:~$ cat /etc/passwd | grep cry0l1t3 | cut -d":" -f1  
cry0l1t3
```

2. The usernames.

```
cat /etc/passwd | cut -d":" -f1
```

```
htb-student@nixfund:~$ cat /etc/passwd | cut -d":" -f1  
root  
daemon  
bin  
sys  
sync  
games  
man
```

3. The username cry0l1t3 and his UID.

```
cat /etc/passwd | grep cry0l1t3 | cut -d":" -f1,3
```

```
htb-student@nixfund:~$ cat /etc/passwd | grep cry0l1t3 | cut -d":" -f1,3  
cry0l1t3:1001
```

4. The username cry0l1t3 and his UID separated by a comma (,).

```
cat /etc/passwd | grep cry0l1t3 | cut -d":" -f1,3 | tr ":" ",,"
```

```
htb-student@nixfund:~$ cat /etc/passwd | grep cry0l1t3 | cut -d":" -f1,3 | tr ":" ",,"  
cry0l1t3,1001  
htb-student@nixfund:~$
```

5. The username cry0l1t3, his UID, and the set shell separated by a comma (,).

```
cat /etc/passwd | grep cry0l1t3 | awk -F":" '{print $1, $3, $NF}' |
tr " " ",,"
```

```
htb-student@nixfund:~$ cat /etc/passwd | grep cry0l1t3 | awk -F":" '{print $1, $3, $NF}' | tr " " ",,"
cry0l1t3,1001,/bin/bash
```

6. All usernames with their UID and set shells separated by a comma (,).

```
cat /etc/passwd | awk -F":" '{print $1, $3 , $NF}' | tr " " ",,"
```

```
htb-student@nixfund:~$ cat /etc/passwd | awk -F":" '{print $1, $3 , $NF}' | tr " " ",,"
root,0,/bin/bash
daemon,1,/usr/sbin/nologin
bin,2,/usr/sbin/nologin
sys,3,/usr/sbin/nologin
sync,4,/bin/sync
```

7. All usernames with their UID and set shells separated by a comma (,) and exclude the ones that contain nologin or false.

```
cat /etc/passwd | grep -v nologin | awk -F":" '{print $1, $3 , $NF}'
| tr " " ",,"
```

```
htb-student@nixfund:~$ cat /etc/passwd | grep -v nologin | awk -F":" '{print $1, $3 , $NF}' | tr " " ",,"
root,0,/bin/bash
sync,4,/bin/sync
lxd,105,/bin/false
pollinate,109,/bin/false
mrb3n,1000,/bin/bash
cry0l1t3,1001,/bin/bash
htb-student,1002,/bin/bash
mysql,116,/bin/false
htb-student@nixfund:~$
```

8. All usernames with their UID and set shells separated by a comma (,) and exclude the ones that contain nologin and count all lines of the filtered output.

```
cat /etc/passwd | grep -v nologin | awk -F":" '{print $1, $3 , $NF}'
| tr " " ",," | wc -l
```

```
htb-student@nixfund:~$ cat /etc/passwd | grep -v nologin | awk -F":" '{print $1, $3 , $NF}' | tr " " ",," | wc -l
8
```


QUESTION 1

How many services are listening on the target system on all interfaces? (Not on localhost and IPv4 only)

Para responder esta pregunta debemos aprender y recordar una idea: siempre que se trate de escuchar, hablamos de comunicación y si hablamos de comunicación hablamos de red; red = network por lo tanto nuestro comando nace desde ahí.

Network STATistics = *netstat*

Los flags más usados:

Comando	Función
<i>netstat -a</i>	Muestra todas las conexiones (activas y en escucha).
<i>netstat -t</i>	Filtra y muestra solo conexiones TCP .
<i>netstat -u</i>	Filtra y muestra solo conexiones UDP .
<i>netstat -l</i>	Muestra solo los puertos que están escuchando (LISTEN).
<i>netstat -n</i>	Muestra todo en números (IPs y puertos, sin nombres como <i>localhost</i>).
<i>sudo netstat -p</i>	Muestra el programa/PID que usa la conexión (requiere sudo).
<i>netstat -tulnp</i>	Un combo muy utilizado

Ya con esta información podemos armar la primera parte de nuestro comando. Dado que nos piden verificar en todas las interfaces probaremos con *netstat -tul*

```
htb-student@nixfund:~$ netstat -tul
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 localhost:mysql         0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:netbios-ssn     0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:pop3            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:imap2          0.0.0.0:*               LISTEN
tcp        0      0 localhost:domain       0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:ssh            0.0.0.0:*               LISTEN
tcp        0      0 localhost:smtp         0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:microsoft-ds    0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:imaps           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:pop3s            0.0.0.0:*               LISTEN
tcp6       0      0 [::]:netbios-ssn       [::]:*                  LISTEN
tcp6       0      0 [::]:pop3              [::]:*                  LISTEN
tcp6       0      0 [::]:imap2             [::]:*                  LISTEN
tcp6       0      0 [::]:http              [::]:*                  LISTEN
tcp6       0      0 [::]:ftp               [::]:*                  LISTEN
tcp6       0      0 [::]:ssh               [::]:*                  LISTEN
tcp6       0      0 ip6-localhost:smtp     [::]:*                  LISTEN
tcp6       0      0 [::]:microsoft-ds      [::]:*                  LISTEN
tcp6       0      0 [::]:imaps             [::]:*                  LISTEN
tcp6       0      0 [::]:pop3s             [::]:*                  LISTEN
udp        0      0 localhost:domain       0.0.0.0:*               LISTEN
udp        0      0 0.0.0.0:bootpc         0.0.0.0:*               LISTEN
udp        0      0 10.129.255.2:netbios-ns 0.0.0.0:*               LISTEN
udp        0      0 10.129.45.15:netbios-ns 0.0.0.0:*               LISTEN
udp        0      0 0.0.0.0:netbios-ns     0.0.0.0:*               LISTEN
udp        0      0 10.129.255.:netbios-dgm 0.0.0.0:*               LISTEN
udp        0      0 10.129.45.1:netbios-dgm 0.0.0.0:*               LISTEN
udp        0      0 0.0.0.0:netbios-dgm    0.0.0.0:*               LISTEN
htb-student@nixfund:~$
```

Como podemos ver en la imagen anterior ya tenemos el listado de servicios que están escuchando, podemos ver que aparecen 3 protocolos, tcp que serían los ipv4, tcp6 que son los ipv6 y udp.

Wait... ¿Por qué aparecen esos udp si con la opción -l del comando se supone que solo debiesen aparecer los que están en modo LISTEN?

La respuesta es muy sencilla, tcp es un protocolo seguro que funciona con el famoso **3 way handshake**, ósea, esos puertos están a la escucha de que se produzca ese saludo para iniciar la conexión. Pero udp es un protocolo que no está pensado para seguridad sino que para ser rápido, como conexiones de videoconferencia o streaming que no importa si se pierden

Pipe y la puerta del reino de Linux | Sección 3: Workflow

algunos bits de información en el camino, lo que importa es que llegue, entonces esos puertos o servicios no están realmente en modo escucha, ellos simplemente están abiertos, no hay un estado formal de LISTEN para ellos y por eso aparecen en la lista más allá de la opción **-l**

Entonces *¿qué debemos hacer para que no aparezcan como lo pide la pregunta (IPv4 only)?* Si pensaste filtrarlos, estas en lo correcto. *¿Y cómo filtramos?* Con grep.

Lo primero que haremos será darle una mirada a **man grep**

```
-i, --ignore-case
    Ignore case distinctions, so that characters that differ only in case match each other.

-v, --invert-match
    Invert the sense of matching, to select non-matching lines.

-w, --word-regexp
    Select only those lines containing matches that form whole words. The test is that the matching substring must either be at the beginning of the line, or preceded by a non-word constituent character. Similarly, it must be either at the end of the line or followed by a non-word constituent character. Word-constituent characters are letters, digits, and the underscore. This option has no effect if -x is also specified.

-x, --line-regexp
    Select only those matches that exactly match the whole line. For a regular expression pattern, this is like parenthesizing the pattern and then surrounding it with ^ and $.
```

Vemos un par de opciones que nos pueden servir. Una es **-w**, que sirve para que el filtro responda a una palabra “exacta” y la otra es **-v** que sirve para excluir un término.


```
netstat -tul | grep -w LISTEN
```

```
htb-student@nixfund:~$ netstat -tul | grep -w LISTEN
tcp        0      0 localhost:mysql      0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:netbios-ssn  0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:pop3         0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:imap2        0.0.0.0:*             LISTEN
tcp        0      0 localhost:domain    0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:ssh         0.0.0.0:*             LISTEN
tcp        0      0 localhost:smtp      0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:microsoft-ds 0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:imaps         0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:pop3s         0.0.0.0:*             LISTEN
tcp6       0      0 [::]:netbios-ssn    [::]:*               LISTEN
tcp6       0      0 [::]:pop3           [::]:*               LISTEN
tcp6       0      0 [::]:imap2          [::]:*               LISTEN
tcp6       0      0 [::]:http           [::]:*               LISTEN
tcp6       0      0 [::]:ftp            [::]:*               LISTEN
tcp6       0      0 [::]:ssh            [::]:*               LISTEN
tcp6       0      0 ip6-localhost:smtp  [::]:*               LISTEN
tcp6       0      0 [::]:microsoft-ds   [::]:*               LISTEN
tcp6       0      0 [::]:imaps          [::]:*               LISTEN
tcp6       0      0 [::]:pop3s          [::]:*               LISTEN
```

Y podemos ver que salieron las conexiones udp.

Importante: En ciberseguridad no es recomendable aplicar un filtro LISTEN como este, debido a que podemos dejar fuera una conexión por **udp** y por eso se recomienda el uso de `netstat -tulnp`, en este caso solo lo utilizamos para responder a la parte del paréntesis de la pregunta (**Not on localhost and IPv4 only**).

Seguiremos filtrando para quedar solo con IPv4 por lo que localhost e IPv6 (tcp6) también debe desaparecer.

```
netstat -tul | grep -w LISTEN | grep -v tcp6 | grep -v localhost
```

```
htb-student@nixfund:~$ netstat -tul | grep -w LISTEN | grep -v tcp6 | grep -v localhost
tcp        0      0 0.0.0.0:netbios-ssn  0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:pop3         0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:imap2        0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:ssh         0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:microsoft-ds 0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:imaps         0.0.0.0:*             LISTEN
tcp        0      0 0.0.0.0:pop3s         0.0.0.0:*             LISTEN
htb-student@nixfund:~$
```

Pipe y la puerta del reino de Linux | Sección 3: Workflow

Como lo que nos están pidiendo es una cantidad (*How many services...*) comenzaremos a utilizar un nuevo comando en vez de nuestro `grep -c`, me refiero a `wc`.

`wc` = **W**ord **C**ount (contar palabras).

Es una herramienta de Linux que sirve para contar los elementos de un texto o del resultado de un comando.

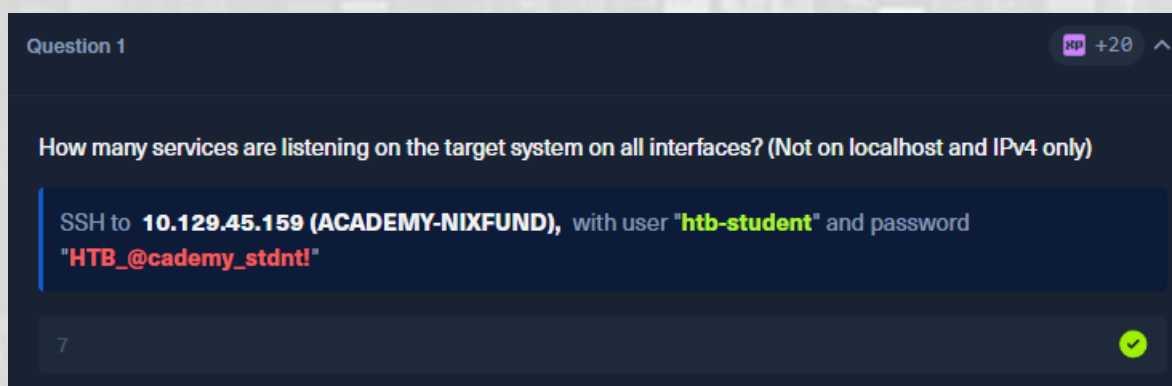
Opciones/flags más usados:

- `wc -l`: Cuenta **líneas** (el más usado en ciberseguridad para contar resultados/puertos).
- `wc -w`: Cuenta **palabras**.
- `wc -c`: Cuenta **caracteres** (bytes).

Le pasas texto y te dice cuántas líneas, palabras o letras tiene.

```
netstat -tul | grep -w LISTEN | grep -v tcp6 | grep -v localhost | wc -l
```

```
htb-student@nixfund:~$ netstat -tul | grep -w LISTEN | grep -v tcp6 | grep -v localhost | wc -l
7
```



ES MUY IMPORTANTE resaltar que la pregunta nos coloca en un escenario forzado y poco probable en la práctica, en escenarios reales no es bueno aplicar el filtro `grep -w LISTEN` ya que esto deja fuera conexiones udp y en ciberseguridad no podemos dejar fuera una conexión cuando estamos analizando métodos de conexión desde y hacia nuestro sistema.

Pipe y la puerta del reino de Linux | Sección 3: Workflow

En este tipo de escenario real, hay una alternativa moderna y recomendada que reemplaza a **netstat** y me refiero a **ss**

El comando **ss** (Socket Statistics) es mucho más rápido ya que obtiene la información directamente del kernel a diferencia de **netstat** que lee los archivos pesados del directorio.

Por este motivo en un escenario real lo que debiese utilizarse es

ss -tuln

independiente si se trata de una distribución Debian o Red Hat.

```
htb-student@nixfund:~$ ss -tuln
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*
udp	UNCONN	0	0	0.0.0.0:68	0.0.0.0:*
udp	UNCONN	0	0	10.129.255.255:137	0.0.0.0:*
udp	UNCONN	0	0	10.129.45.231:137	0.0.0.0:*
udp	UNCONN	0	0	0.0.0.0:137	0.0.0.0:*
udp	UNCONN	0	0	10.129.255.255:138	0.0.0.0:*
udp	UNCONN	0	0	10.129.45.231:138	0.0.0.0:*
udp	UNCONN	0	0	0.0.0.0:138	0.0.0.0:*
tcp	LISTEN	0	100	0.0.0.0:110	0.0.0.0:*
tcp	LISTEN	0	100	0.0.0.0:143	0.0.0.0:*
tcp	LISTEN	0	128	127.0.0.53%lo:53	0.0.0.0:*
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*
tcp	LISTEN	0	100	127.0.0.1:25	0.0.0.0:*
tcp	LISTEN	0	50	0.0.0.0:445	0.0.0.0:*
tcp	LISTEN	0	100	0.0.0.0:993	0.0.0.0:*
tcp	LISTEN	0	100	0.0.0.0:995	0.0.0.0:*
tcp	LISTEN	0	80	127.0.0.1:3306	0.0.0.0:*
tcp	LISTEN	0	50	0.0.0.0:139	0.0.0.0:*
tcp	LISTEN	0	100	[::]:110	[::]:*
tcp	LISTEN	0	100	[::]:143	[::]:*
tcp	LISTEN	0	128	*:80	*:*
tcp	LISTEN	0	32	*:21	*:*
tcp	LISTEN	0	128	[::]:22	[::]:*
tcp	LISTEN	0	100	[::1]:25	[::]:*
tcp	LISTEN	0	50	[::]:445	[::]:*
tcp	LISTEN	0	100	[::]:993	[::]:*
tcp	LISTEN	0	100	[::]:995	[::]:*
tcp	LISTEN	0	50	[::]:139	[::]:*

QUESTION 2

Determine what user the ProFTPD server is running under. Submit the username as the answer.

Antes de responder hay un comando que debemos entender y agregar a nuestro repertorio, **ps**.

El comando **ps** (Process Status) nos permite ver que procesos se están ejecutando en el sistema en el momento en que lo ejecutamos.

Como la pregunta dice *“is running”* entonces estamos hablando de un proceso.

Una línea de comando que es el standard universal en ciberseguridad que se utiliza para revisar todo el sistema es

ps aux

- **a** : Todos los procesos con una terminal (TTY).
- **u** : Formato orientado al usuario (muestra columnas de CPU, memoria, etc.).
- **x** : Procesos que no tienen una terminal asociada (servicios en segundo plano conocidos como DAEMONS).

Podemos usar esta opción y aplicar un filtro como lo hemos venido haciendo hasta ahora.

ps aux | grep ftpd

```
htb-student@nixfund:~$ ps aux | grep ftpd
proftpd  1884  0.0  0.1 126444 3668 ?        Ss   09:55   0:00 proftpd: (accepting connections)
htb-stu+ 6551  0.0  0.0 13144 1100 pts/0    S+   11:55   0:00 grep --color=auto ftpd
htb-student@nixfund:~$
```

Podemos ver que en la salida nos envía 2 líneas, la primera es el usuario proftpd corriendo el servicio proftpd y la segunda somos nosotros

Pipe y la puerta del reino de Linux | Sección 3: Workflow

corriendo el comando **grep**. Si bien con eso tenemos para la respuesta, lo cierto es que podemos limpiar más la salida con un método alternativo.

Dado que nos piden determinar solo el usuario podemos utilizar

```
ps -o user= -C proftpd
```

Comando/flag	Explicación
ps	Comando base.
-o	Output, con este elegimos que columnas queremos ver en la salida.
-o user=	Muestra solo la columna user y con el signo = elimina título de arriba.
-C proftpd	Filtra el proceso por el nombre exacto proftpd

Dicho de otra forma, nos muestra solo el usuario que corre ProFTPD.

```
htb-student@nixfund:~$ ps -o user= -C proftpd
proftpd
htb-student@nixfund:~$ |
```

Question 2 XP +20

Determine what user the ProFTPD server is running under. Submit the username as the answer.

✓

QUESTION 3

Use cURL from your Pwnbox (not the target machine) to obtain the source code of the "https://www.inlanefreight.com" website and filter all unique paths (https://www.inlanefreight.com/directory" or "/another/directory") of that domain. Submit the number of these paths as the answer.

Antes de responder hay algunos conceptos y comandos que debemos saber. Como menciona el enunciado aparece **curl**.

Como herramienta de línea de comando, **curl** (Client URL) nos permite transferir datos desde y hacia un servidor directamente desde la terminal o dicho en corto es un navegador web ultra rápido en formato de texto para nuestra terminal.

Entre sus flags más utilizados tenemos:

- **curl http://sitio.com**: Descarga el código HTML de la página y lo muestra en la terminal.
- **curl -s** (Silent): Modo silencioso. No muestra barras de progreso ni errores de conexión.
- **curl -I** (Include): Trae únicamente las **cabeceras** HTTP del servidor (útil para ver versiones de software o cookies).
- **curl -O** (Output): Descarga el archivo manteniendo su nombre original.
- **curl -X POST**: Cambia el método de petición (por defecto es GET) para enviar datos a un servidor.

Para poder realizar este curl nos dice que podemos hacerlo desde la terminal Pwnbox pero yo lo haré desde Kali Linux. Es lo mismo. Si ejecuto **curl -s https://www.inlanefreight.com** podemos ver cómo nos envía todo el código de la página.


```

(ozone@kali)-[~]
$ curl -s https://www.inlanefreight.com
<!DOCTYPE html>
<html lang="en-US">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="profile" href="http://gmpg.org/xfn/11">
<title>Inlanefreight &#8211; Protected by Wordfence</title>
<link rel='dns-prefetch' href='//fonts.googleapis.com' />
<link rel='dns-prefetch' href='//s.w.org' />
<link rel="alternate" type="application/rss+xml" title="Inlanefreight &#8211; Feed" href="https://www.inlanefreight.com/index.php/feed/" />
<link rel="alternate" type="application/rss+xml" title="Inlanefreight &#8211; Comments Feed" href="https://www.inlanefreight.com/index.php/comments/feed/" />
<script type="text/javascript">
    window._wpemojiSettings = {"baseUrl":"https://s.w.org/images/core/emoji/13.0.1/72x72/",
    "ext":".png","svgUrl":"https://s.w.org/images/core/emoji/13.0.1/svg/",
    "svgExt":".svg","source":{"concatemoji":"https://www.inlanefreight.com/wp-includes/js/wp-emoji-release.min.js?ver=5.6.17"}};
    !function(e,a,t){var n,r,o,i=a.createElement("canvas"),p=i.getContext("2d");function s(e,t){var a=String.fromCharCode;p.clearRect(0,0,i.width,i.height),p.fillText(a.apply(this,e),0,0);e=i.toDataURL();return p.clearRect(0,0,i.width,i.height),p.fillText(a.apply(this,t),0,0),e=i.toDataURL(),function c(e){var t=a.createElement("script");t.src=e,t.defer=t.type="text/javascript",a.getElementsByTagName("head")[0].appendChild(t)}for(o=Array("flag","emoji"),t.supports={everything:!0,everythingExceptFlag:!0},r=0;r<o.length;r++)t.supports[o[r]]=function(e){if(!p||!p.fillText)return!1;switch(p.textBaseline="top",p.font="600 32px Arial",e){case"flag":return s([127987,65039,8205,9895,65039],[127987,65039,8203,9895,65039])?!1:s([55356,56826,55356,56819],[55356,56826,8203,55356,56819])&&s([55356,57332,56128,56423,56128,56418,56128,56421,56128,56430,56128,56423,56128,56447],[55356,57332,8203,56128,56423,8203,56128,56418,8203,56128,56421,8203,56128,56430,8203,56128,56423,8203,56128,56447]);case"emoji":return s([55357,56424,8205,55356,57212],[55357,56424,8203,55356,57212])};return!1}(o[r]),t.supports.everything=t.supports.everything&&t.supports[o[r]],"flag"===o[r]&&(t.supports.everythingExceptFlag=t.supports.everythingExceptFlag&&t.supports[o[r]]);t.supports.everythingExceptFlag=t.supports.everythingExceptFlag&&!t.supports.flag,t.DOMReady=!1,t.readyCallback=function(){t.DOMReady=!0},t.supports.everything||(n=function(){t.readyCallback()},a.addEventListener("DOMContentLoaded",n,!1),e.addEventListener("load",n,!1)):(e.attachEvent("onload",n),a.attachEvent("onreadystatechange",function(){"complete"===a.readyState&&t.readyCallback()})),(n=t.source||{}).concatemoji?c(n.concatemoji):n.wpemoji&&n.twemoji&&c(n.twemoji,c(n.wpemoji))}(window,document,window._wpemojiSettings);
</script>

```

Por este motivo filtraremos con grep y expresiones regulares.

Expresiones regulares... Qué es eso?!?!?

Explicado en corto son patrones de texto que permiten buscar, filtrar o reemplazar palabras o caracteres dentro de un archivo o comando.

También tenemos las **expresiones regulares AVANZADAS**.

Estas son una versión extendida de la anterior que añade operadores lógicos más potentes para hacer búsquedas complejas sin la necesidad de tener que usar caracteres de escape (/) que vuelven el código ilegible.

Estas “avanzadas” las llamamos con **grep -E** o también con **egrep**. En la práctica casi todos los profesionales de ciberseguridad utilizan las expresiones regulares avanzadas.

Pipe y la puerta del reino de Linux | Sección 3: Workflow

```
curl -s https://www.inlanefreight.com | grep -oE  
"https://www.inlanefreight.com[^'\"]*"
```

```
(ozone@kali)-[~]  
$ curl -s https://www.inlanefreight.com | grep -oE "https://www.inlanefreight.com[^'\"]*"
https://www.inlanefreight.com/index.php/feed/
https://www.inlanefreight.com/index.php/comments/feed/
https://www.inlanefreight.com/wp-includes/css/dist/block-library/style.min.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/bootstrap.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/style.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/colors/default.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/jquery.smartmenus.bootstrap.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/owl.carousel.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/owl.transitions.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/font-awesome.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/animate.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/magnific-popup.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/bootstrap-progressbar.min.css?ver=5.6.17
https://www.inlanefreight.com/wp-includes/js/jquery/jquery.min.js?ver=3.5.1
https://www.inlanefreight.com/wp-includes/js/jquery/jquery-migrate.min.js?ver=3.3.2
https://www.inlanefreight.com/wp-content/themes/ben_theme/js/navigation.js?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/js/bootstrap.min.js?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/js/jquery.smartmenus.js?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/js/jquery.smartmenus.bootstrap.js?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/js/owl.carousel.min.js?ver=5.6.17
https://www.inlanefreight.com/index.php/wp-json/
https://www.inlanefreight.com/index.php/wp-json/wp/v2/pages/7
https://www.inlanefreight.com/xmlrpc.php?rsd
https://www.inlanefreight.com/wp-includes/wlwmanifest.xml
```

- `curl -s https://...`
Descarga el código HTML de la web en modo silencioso.
- `grep -oE "https://..."`
 - `-o` Extrae **solo** el texto que coincide con la búsqueda (no la línea completa).
 - `-E` Activa expresiones regulares avanzadas.
 - `[^'\"]*` Busca todo lo que empiece con la URL hasta que encuentre una comilla simple (') o doble ("), cortando ahí el enlace de forma limpia.

Ahora solo debemos asegurarnos de que no aparezcan duplicados, para esto usaremos el comando `sort` (ordenar alfabéticamente) con la flag `-u` (unique).

Pipe y la puerta del reino de Linux | Sección 3: Workflow

```
curl -s https://www.inlanefreight.com | grep -oE
"https://www.inlanefreight.com[^\"]*" | sort -u
```

```
(ozone@kali)-[~]
$ curl -s https://www.inlanefreight.com | grep -oE "https://www.inlanefreight.com[^\"]*" | sort -u
https://www.inlanefreight.com/
https://www.inlanefreight.com/index.php/about-us/
https://www.inlanefreight.com/index.php/career/
https://www.inlanefreight.com/index.php/comments/feed/
https://www.inlanefreight.com/index.php/contact/
https://www.inlanefreight.com/index.php/feed/
https://www.inlanefreight.com/index.php/news/
https://www.inlanefreight.com/index.php/offices/
https://www.inlanefreight.com/index.php/wp-json/
https://www.inlanefreight.com/index.php/wp-json/oembed/1.0/embed?url=https%3A%2F%2Fwww.inlanefreight.com%2F
https://www.inlanefreight.com/index.php/wp-json/oembed/1.0/embed?url=https%3A%2F%2Fwww.inlanefreight.com%2F
https://www.inlanefreight.com/index.php/wp-json/wp/v2/pages/7
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/animate.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/bootstrap.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/bootstrap-progressbar.min.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/colors/default.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/font-awesome.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/jquery.smartmenus.bootstrap.css?ver=5.6.17
https://www.inlanefreight.com/wp-content/themes/ben_theme/css/magnific-popup.css?ver=5.6.17
```

Listo, tan solo nos queda saber cuántos son para responder por lo que agregamos `wc -l`

```
curl -s https://www.inlanefreight.com | grep -oE
"https://www.inlanefreight.com[^\"]*" | sort -u | wc -l
```

```
(ozone@kali)-[~]
$ curl -s https://www.inlanefreight.com | grep -oE "https://www.inlanefreight.com[^\"]*" | sort -u | wc -l
34
```

Question 3
+1
XP +20

Use cURL from your Pwnbox (not the target machine) to obtain the source code of the "https://www.inlanefreight.com" website and filter all unique paths (https://www.inlanefreight.com/directory" or "/another/directory") of that domain. Submit the number of these paths as the answer.

34

IMPORTANTE.

Cabe señalar que las siguientes secciones REGULAR EXPRESSIONS y PERMISSION MANAGEMENT no incluyen QUESTIONS pero es muy recomendable realizar las tareas de practica que ayudan precisamente para entender el uso de expresiones regulares y la asignación de permisos.

Regular Expressions

Para estos ejercicios utilizar el archivo `/etc/ssh/sshd_config` de la máquina que se conecta por ssh.

1. Show all lines that do not contain the # character.

```
grep -vE "#|^$" /etc/ssh/sshd_config
```

```
htb-student@nixfund:~$ grep -vE "#|^$" /etc/ssh/sshd_config
PermitRootLogin no
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2
PasswordAuthentication yes
PermitEmptyPasswords no
ChallengeResponseAuthentication no
UsePAM yes
X11Forwarding yes
PrintMotd no
AcceptEnv LANG LC_*
Subsystem sftp /usr/lib/openssh/sftp-server
PasswordAuthentication yes
htb-student@nixfund:~$
```

2. Search for all lines that contain a word that starts with Permit.

```
grep "^Permit" /etc/ssh/sshd_config
```

```
htb-student@nixfund:~$ grep "^Permit" /etc/ssh/sshd_config
PermitRootLogin no
PermitEmptyPasswords no
htb-student@nixfund:~$
```

3. Search for all lines that contain a word ending with Authentication.

```
grep -E "Authentication$" /etc/ssh/sshd_config
```

```
htb-student@nixfund:~$ grep -E "Authentication$" /etc/ssh/sshd_config
# HostbasedAuthentication
# PAM authentication, then enable this but set PasswordAuthentication
htb-student@nixfund:~$
```

4. Search for all lines containing the word Key.

```
grep -E "([a-zA-Z]|^)key([a-zA-Z]|$)" /etc/ssh/sshd_config
```

```
htb-student@nixfund:~$ grep -E "([a-zA-Z]|^)key([a-zA-Z]|$)" /etc/ssh/sshd_config
#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key
htb-student@nixfund:~$
```

5. Search for all lines beginning with Password and containing yes.

```
grep -E "^Password.*yes$" /etc/ssh/sshd_config
```

```
htb-student@nixfund:~$ grep -E "^Password.*yes$" /etc/ssh/sshd_config
PasswordAuthentication yes
PasswordAuthentication yes
htb-student@nixfund:~$
```

6. Search for all lines that end with yes.

```
grep -E "yes$" /etc/ssh/sshd_config
```

```
htb-student@nixfund:~$ grep -E "yes$" /etc/ssh/sshd_config
#StrictModes yes
PubkeyAuthentication yes
#IgnoreRhosts yes
PasswordAuthentication yes
#KerberosOrLocalPasswd yes
#KerberosTicketCleanup yes
#GSSAPICleanupCredentials yes
#GSSAPIStrictAcceptorCheck yes
UsePAM yes
#AllowAgentForwarding yes
#AllowTcpForwarding yes
X11Forwarding yes
#X11UseLocalhost yes
#PermitTTY yes
#PrintLastLog yes
#TCPKeepAlive yes
PasswordAuthentication yes
htb-student@nixfund:~$
```

Permission Management

Este capítulo en particular no trae preguntas ni ejercicios de práctica pero me parece sensato inventarse una pequeña practica para poder entender **SUID**, **SGID** y **StickyBit**.

Personalmente creo que el video del enlace a continuación lo explica de manera sublime, mejor que cualquier manual o inteligencia artificial. Recomendando hacer esos pequeños ejercicios que realiza el autor del video.

<https://www.youtube.com/watch?v=OnoQahwN9yo>



Sección 4: System Management

User Management

QUESTION 1

Which option needs to be set to create a home directory for a new user using "useradd" command?

Básicamente nos están preguntando por una flag del comando `useradd`, por lo que usaremos `man useradd`. El punto es que como el manual de este comando `useradd` es tan grande, encontrar lo que estamos buscando puede ser difícil. Por esto, si no lo sabías, cuando estes dentro de la salida del comando `man useradd` vas a presionar `/` y resaltará el texto que estas buscando lo que hace más sencilla la búsqueda.

`/home directory`

```
-c, --comment COMMENT
    Any text string. It is generally a short
    user's full name.

-d, --home-dir HOME_DIR
/home directory
```

```
-m, --create-home
    Create the user's home directory if it does not exist. The files and directories contained in the skeleton directory (which can be defined with the -k option) will be copied to the home directory.

    By default, if this option is not specified and CREATE_HOME is not enabled, no home directories are created.

    The directory where the user's home directory is created must exist and have proper SELinux context and permissions. Otherwise the user's home directory cannot be created or accessed.
```

Al avanzar página podemos ver cómo nos resalta la búsqueda y en este caso nos da la respuesta `-m`.

¿Porque no nos sirve grep?

man useradd | grep "home directory"

```
(ozone@kali)-[~]
$ man useradd | grep "home directory"
update system files and may also create the new user's home directory and copy initial files.
account name to define the home directory.
The skeleton directory, which contains files and directories to be copied in the user's home directory, when the
home directory is created by useradd.
home directory.
Create the user's home directory if it does not exist. The files and directories contained in the skeleton di-
rectory (which can be defined with the -k option) will be copied to the home directory.
The directory where the user's home directory is created must exist and have proper SELinux context and permis-
sions. Otherwise the user's home directory cannot be created or accessed.
Do not create the user's home directory, even if the system wide setting from /etc/login.defs (CREATE_HOME) is
/etc/login.defs (CREATE_HOME). You have to specify the -m options if you want a home directory for a system ac-
cesses the path prefix for a new user's home directory. The user's name will be affixed to the end of BASE_DIR to
form the new user's home directory name, if the -d option is not used when creating a new account.
Indicate if a home directory should be created by default for new users.
useradd and newusers use this to set the mode of the home directory they create.
Defines the location of the users mail spool files relatively to their home directory.
useradd and newusers use this mask to set the mode of the home directory they create if HOME_MODE is not set.
can't create home directory

(ozone@kali)-[~]
$
```

Si bien resalta lo que estamos buscando, no tenemos idea de a que flag corresponde. Por eso lo mejor es la opción anterior con /.

Ahora, el problema con el comando **useradd** es que si se nos olvida añadir la flag **-m** para crear el directorio, el usuario no tendrá directorio de inicio.

Para consultar los usuarios del sistema podemos usar **cat /etc/passwd**

```
(ozone@kali)-[~]
$ sudo useradd ruben

(ozone@kali)-[~]
$ cat /etc/passwd | tail
geoclue:x:977:977::/var/lib/geoclue:/usr/sbin/nologin
lightdm:x:118:121:Light Display Manager:/var/lib/lightdm:/bin/false
polkitd:x:976:976:User for polkitd:/usr/sbin/nologin
rtkit:x:975:975:RealtimeKit:/proc:/usr/sbin/nologin
colord:x:974:974:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
pcscd:x:973:973:PC/SC Smart Card Daemon:/usr/sbin/nologin
ozone:x:1000:1000:ozone,,:/home/ozone:/bin/zsh
_dhcpd:x:972:972:DHCP Client Daemon:/var/lib/dhcpd:/sbin/nologin
stunnel:x:971:971:stunnel service system account:/var/run/stunnel:/usr/sbin/nologin
ruben:x:1001:1001::/home/ruben:/bin/sh

(ozone@kali)-[~]
$ cd /home/ruben
cd: no such file or directory: /home/ruben
```

Pipe y la puerta del reino de Linux | Sección 4: System Management

Lo cierto es que actualmente **useradd** se considera de bajo nivel, tenemos una alternativa que no solo crea el usuario, sino que además permite asignar una password y crea el directorio de inicio del usuario en un mismo comando. Me refiero a **adduser** (exacto!! Al revés).

```
(ozone@kali)-[~]  
$ sudo adduser ruben  
New password:  
Retype new password:  
passwd: password updated successfully  
Changing the user information for ruben  
Enter the new value, or press ENTER for the default  
    Full Name []: Ruben  
    Room Number []:  
    Work Phone []:  
    Home Phone []:  
    Other []:  
Is the information correct? [Y/n] Y
```

```
(ruben@kali)-[~]  
$ cat /etc/passwd | tail  
geoclue:x:977:977::/var/lib/geoclue:/usr/sbin/nologin  
lightdm:x:118:121:Light Display Manager:/var/lib/lightdm:/bin/false  
polkitd:x:976:976:User for polkitd:/usr/sbin/nologin  
rtkit:x:975:975:RealtimeKit:/proc:/usr/sbin/nologin  
colord:x:974:974:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin  
pcscd:x:973:973:PC/SC Smart Card Daemon:/usr/sbin/nologin  
ozone:x:1000:1000:ozone,,,:/home/ozone:/bin/zsh  
_dhcpcd:x:972:972:DHCP Client Daemon:/var/lib/dhcpcd:/sbin/nologin  
stunnel:x:971:971:stunnel service system account:/var/run/stunnel:/usr/sbin/nologin  
ruben:x:1001:1001:Ruben,,,:/home/ruben:/bin/bash
```

```
(ozone@kali)-[~]  
$ su ruben  
Password:
```

```
(ruben@kali)-[~]  
$ pwd  
/home/ruben
```

La respuesta entonces:

Question 1 HP +20

Which option needs to be set to create a home directory for a new user using "useradd" command?

-m ✓

QUESTION 2

Which option needs to be set to lock a user account using the "usermod" command? (long version of the option)

La lógica para responder es la misma, `man usermod`, iniciamos la búsqueda con `/` y buscamos por el termino `lock` y la respuesta a lo que nos piden es `--lock`

```
-L, --lock
Lock a user's password. This puts a '!' in front of the encrypted password, effectively disabling the password.
You can't use this option with -p or -U.

Note: if you wish to lock the account (not only access with a password), you should also set the EXPIRE_DATE to 1.
```

Question 2

XP +20 ^

Which option needs to be set to lock a user account using the "usermod" command? (long version of the option)

`--lock`

Pero más allá de quedarnos solo con la respuesta, probemos de que va este comando. Como vemos en las imágenes siguientes, creamos el usuario y su contraseña con `useradd`, luego revisamos que efectivamente este haya sido creado, luego cambiamos de cuenta al nuevo usuario creado con `su ruben` (en este caso) y comprobamos con el comando `whoami`.

```
(ozone@kali)-[~]
$ sudo adduser ruben
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for ruben
Enter the new value, or press ENTER for the default
    Full Name []: Ruben
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y

(ozone@kali)-[~]
$ cat /etc/passwd | tail
geoclue:x:977:977::/var/lib/geoclue:/usr/sbin/nologin
lightdm:x:118:121:Light Display Manager:/var/lib/lightdm:/bin/false
polkitd:x:976:976:User for polkitd:/usr/sbin/nologin
rtkit:x:975:975:RealtimeKit:/proc:/usr/sbin/nologin
colord:x:974:974:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
pcscd:x:973:973:PC/SC Smart Card Daemon:/usr/sbin/nologin
ozone:x:1000:1000:ozone,,,:/home/ozone:/bin/zsh
_dhcpd:x:972:972:DHCP Client Daemon:/var/lib/dhcpd:/sbin/nologin
stunnel:x:971:971:stunnel service system account:/var/run/stunnel:/usr/sbin/nologin
ruben:x:1001:1001:Ruben,,,:/home/ruben:/bin/bash
```

```
(ozone@kali)-[~]
$ su ruben
Password:
(ruben@kali)-[/home/ozone]
$ whoami
ruben
```

A continuación podemos ver como de vuelta desde la cuenta ozone aplicamos el bloqueo de contraseña con la versión corta de **--lock** que es **-L** (**sudo usermod -L ruben**) y cuando volvemos a intentar cambiar de usuario a esa cuenta vemos que la autenticación ha sido bloqueada. Como adicional recomiendo averiguar cómo se desbloquea esa cuenta bloqueada.

```
(ruben@kali)-[/home/ozone]
$ su ozone
Password:
(ozone@kali)-[~]
$ sudo usermod -L ruben

(ozone@kali)-[~]
$ su ruben
Password:
su: Authentication failure
```

QUESTION 3

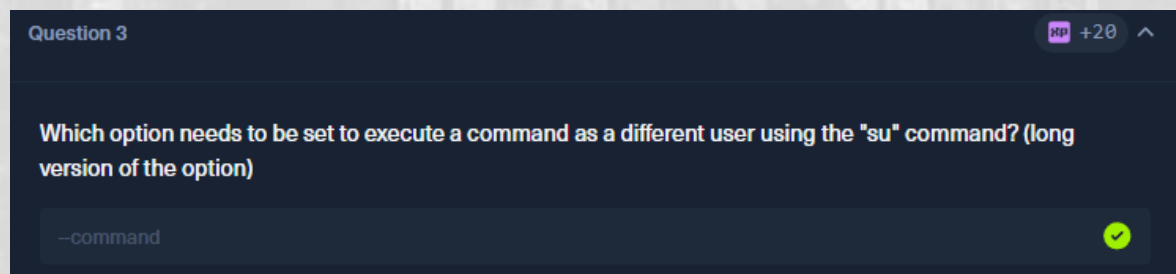
Which option needs to be set to execute a command as a different user using the "su" command? (long version of the option)

Este comando es como jugar a *"Simón manda"*, básicamente lo que hace es ejecutar un comando como si fuésemos otro usuario del sistema de forma transitoria.

Si vamos a `man su` podemos ver que la opción que buscamos es `-c` o en su versión larga `--command`

```
OPTIONS
-c, --command command
    Pass command to the shell with the -c option. Creates a new session via setsid(2). Refer to --session-command to keep the same session.
```

Por lo tanto la respuesta a la pregunta es



Ahora si lo queremos probar, crearemos un nuevo usuario con `adduser` como lo vimos en la QUESTION 1 y lo probaremos enviando un ping a Google como el usuario **ruben** pero desde el usuario **ozone**. Cuando el comando termina, la sesión transitoria se cierra automáticamente.

```
(ozone@kali)-[~]
$ su -c "ping 8.8.8.8" ruben
Password:
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=13.0 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=12.6 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=9.48 ms
^C
Session terminated, killing shell... ..killed.
```


Service and Process Management

QUESTION 1

Use the "systemctl" command to list all units of services and submit the unit name with the description "Load AppArmor profiles managed internally by snapd" as the answer.

Para responder aplicaremos lógicas anteriores. Listaremos y filtraremos lo que nos indica la pregunta.

```
systemctl -l | grep apparmor
```

```
htb-student@nixfund:~$ systemctl -l | grep apparmor
apparmor.service
r initialization
snapd.apparmor.service
pArmor profiles managed internally by snapd
htb-student@nixfund:~$
```

Por lo tanto la respuesta

Question 1  +1  +20 ^

Use the "systemctl" command to list all units of services and submit the unit name with the description "Load AppArmor profiles managed internally by snapd" as the answer.

SSH to **10.129.48.97 (ACADEMY-NIXFUND)**, with user "**htb-student**" and password "**HTB_@cademy_stdnt!**"

snapd.apparmor.service 

Task Scheduling

QUESTION 1

What is the Type of the service of the "dconf.service"?

Lo primero que hay que dejar en claro es que la máquina de HTB si queremos responder a esta pregunta no nos servirá. al conectar por ssh tiene un fallo para responder a esta pregunta, la máquina de HTB no sirvió por dos razones: no tenía el servicio instalado (era un servidor sin entorno gráfico) y además la sesión SSH estaba limitada.

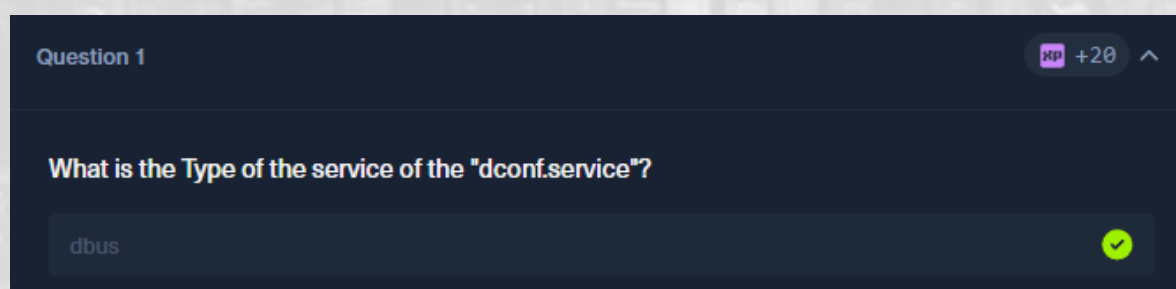
```
htb-student@nixfund:~$ systemctl --user show dconf.service | grep -i type
htb-student@nixfund:~$
```

El mismo comando en mi maquina Kali Linux

```
systemctl --user show dconf.service | grep -i type
```

```
(ozone@kali)-[~]
$ systemctl --user show dconf.service | grep -i type
Type=dbus
ExitType=main
```

Y la respuesta correcta a la pregunta



Question 1 HP +20 ^

What is the Type of the service of the "dconf.service"?

dbus ✓

Pero este es un buen ejercicio para entender un poco más a fondo.

Lo primero. El 99% de las veces que necesitemos comunicarnos con **systemd** utilizaremos **systemctl** que es la herramienta de comando que permite esa comunicación.

¿Y que es systemd?

Es el primer proceso, el administrador central del sistema Linux. Cuando encendemos el equipo o la vm con Linux, el Kernel de Linux al primero que arranca es a **systemd** y por eso recibe el **PID 1**. En las distribuciones de Debian y Redhat adoptaron **systemd** como administrador central del sistema.

¿Pero si **systemd** tiene siempre PID 1 por que cuando realizo **ps aux** me muestra **/sbin/init** en vez **systemd**?

```
htb-student@nixfund:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.1  0.4 225516 9244 ?        Ss   20:15   0:02 /sbin/init maybe-ubiquity
root         2  0.0  0.0      0     0 ?        S    20:15   0:00 [kthreadd]
root         4  0.0  0.0      0     0 ?        T<   20:15   0:00 [kworker/0:0H]
```

Esto se debe a un tema de compatibilidad con sistemas antiguos. Según cuentan los sabios que cuando se adoptó **systemd** como el nuevo estándar, en lugar de borrar el archivo tradicional **/sbin/init**, los desarrolladores lo reemplazaron como un enlace simbólico (acceso directo) a **systemd**. Podemos comprobar esto con los comandos **ls** o **file** como lo muestra la imagen a continuación.

```
htb-student@nixfund:~$ ls -l /sbin/init
lrwxrwxrwx 1 root root 20 Jul  8 2020 /sbin/init -> /lib/systemd/systemd
htb-student@nixfund:~$ file /sbin/init
/sbin/init: symbolic link to /lib/systemd/systemd
htb-student@nixfund:~$
```


¿ Y porque querría comunicarme con **systemd** para revisar **dconf.service**?

Porque es un servicio y como tal corre bajo la “administración” de **systemd** en su modo usuario.

¿Modo usuario? ¿Qué modos existen y cuantos son?

systemd opera en dos modos independientes que no se mezclan.

MODO SISTEMA (**systemctl**)

- ¿Qué controla?

Controla los cimientos de la maquina entera (internet, ssh, discos, Docker, el reloj del sistema)

- ¿Quién manda?

El usuario administrador (root /sudo)

- ¿Cuándo arranca?

Cuando se enciende el equipo antes de que nadie ponga su contraseña

MODO USUARIO (**systemctl --user**)

- ¿Qué controla?

Los caprichos visuales y herramientas de la sesión personal de cada usuario, el audio de los auriculares, el servidor de archivos compartidos del escritorio del usuario, las claves personales y dconf.

- ¿Quién manda?

El usuario normal sin necesidad de sudo

- ¿Cuándo arranca?

Solo cuando se inicia sesión con el usuario y contraseña. Si la sesión se cierra esos servicios se apagan.

La regla de oro para los administradores, si el servicio afecta a todos los usuarios usamos **systemctl**, si afecta solo a tu pantalla o tus archivos **systemctl --user**.

Entonces y en definitiva ¿qué es dconf?

Es simplemente una base de configuraciones, guarda checks y valores de los programas visuales.

- ¿El modo oscuro esta activado? – SI
- ¿El volumen de los auriculares está al 80%? – SI

Si **systemd --user** es el electricista que enciende y mantiene funcionando los auriculares y la pantalla, **dconf** es la libretita donde el electricista anota con que volumen le gusta la música al usuario para que no se olvide cuando se reinicia el sistema.

Toda esta explicación larga nos lleva entonces a formular nuestras líneas de comando para consultar por ese servicio **dconf.service** y porque en realidad es una pregunta más bien teórica que no se puede responder desde la máquina de HTB pero si de mi VM Kali Linux.

Sabemos que debemos consultar con **systemctl** y también aprendimos que como **dconf** es una configuración de usuario y no de sistema hay que agregar el **--user** al comando. Primero consultaremos el estado del servicio, para esto utilizamos la opción **status** y el nombre del servicio que en este caso es **dconf.service**

systemctl --user status dconf.service

```
htb-student@nixfund:~$ systemctl --user status dconf.service
Unit dconf.service could not be found.
htb-student@nixfund:~$
```

Pipe y la puerta del reino de Linux | Sección 4: System Management

El servicio no está corriendo en la maquina como se dijo al inicio. Principalmente por 2 razones, la máquina de HTB no tiene entorno gráfico y además la sesión SSH es limitada en este tipo de máquinas.

Probaré lo mismo pero esta vez en mi maquina Kali Linux.

```
(ozone@kali)-[~]
$ systemctl --user status dconf.service
● dconf.service - User preferences database
   Loaded: loaded (/usr/lib/systemd/user/dconf.service; static)
   Active: active (running) since Tue 2026-06-30 16:25:54 -04; 1h 25min ago
 Invocation: 2769269299f14c1c914acd7ad7823b97
    Docs: man:dconf-service(1)
  Main PID: 1524 (dconf-service)
    Tasks: 4 (limit: 9238)
  Memory: 868K (peak: 2M)
     CPU: 16ms
   CGroup: /user.slice/user-1000.slice/user@1000.service/app.slice/dconf.service
           └─1524 /usr/libexec/dconf-service

Jun 30 16:25:54 kali systemd[1316]: Starting dconf.service - User preferences database ...
Jun 30 16:25:54 kali systemd[1316]: Started dconf.service - User preferences database.
```

Vemos como en esta vm el servicio si está corriendo. Ahora preguntaremos por el tipo del archivo **dconf.service** como pide la pregunta.

Sabemos que es **systemctl --user** pero ahora en vez del **estado** del servicio queremos que nos **muestre (show)** el servicio **dconf.service** y sobre esa salida aplique un filtro con **grep -i** (para ignorar mayúsculas y minúsculas) para **type**.

```
systemctl --user show dconf.service | grep -i type
```

```
(ozone@kali)-[~]
$ systemctl --user show dconf.service | grep -i type
Type=dbus
ExitType=main
```

Y esa es la forma de obtener la respuesta.

¿Qué son los servicios D-Bus?

D-Bus es el *"sistema de mensajería"* interna de Linux. Es un canal de comunicación que permite a diferentes programas y servicios hablar entre sí y pasarse datos en tiempo real (por ejemplo, que el reproductor de música le avise a la pantalla que cambió de canción).

¿Cuántos tipos de buses hay?

Solo hay **2 tipos** principales en cualquier sistema Linux:

- **Bus de sistema:** Un canal único para todo el equipo. Lo usan servicios críticos (como la batería, la red o los discos) para enviar alertas a todo el sistema.
- **Bus de sesión:** Un canal privado por cada usuario que inicia sesión. Lo usan tus aplicaciones visuales (como dconf, reproductores de música o el entorno de escritorio).

Network Services

QUESTION 1

Find a way to start a simple HTTP server inside Pwnbox or your local VM using "npm". Submit the command that starts the web server on port 8080 (use the short argument to specify the port number).

Lo cierto es que para responder a esta pregunta tuve que recurrir a internet ya que hasta ese momento nunca había escuchado de levantar un servidor por **npm**, si había levantado un servidor Apache o uno con Python para compartir archivos pero de **npm** nada.

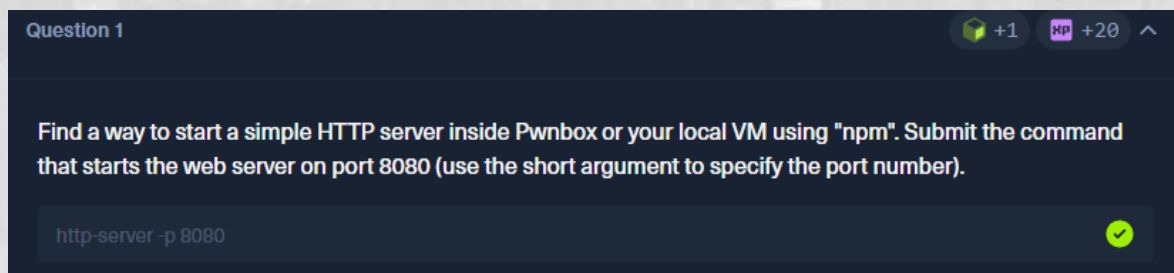
Lo que aprendí fue valioso así que trataré de explicarlo en mis términos sin alargarme demasiado.

Node Package Manager (npm) es el gestor de paquetes de Node.js, dicho de otra forma es como el apt o el dpkg para node y javascript.

Para responder a lo solicitado el orden sería el siguiente, en caso de que el sistema no lo tenga:

1. Instalar con sudo `apt install npm`
2. Instalamos el paquete del servidor con `sudo npm install -g http-server` o la versión corta `npm i -g`
3. Ejecutamos el servidor con `http-server -p 8080`

Por lo tanto la respuesta es:



¿Pero cuál deberíamos utilizar? ¿Apache, python o npm?

Depende el caso.

Si estamos trabajando en un entorno real de producción el estándar sería ir por **Nginx** o **Apache** y tener a la vista uno que está entrando fuerte llamado **Caddy**.

Para un entorno pequeño de red casera en el que nos pasamos archivos de un equipo a otro, incluyendo sistemas operativos distintos (Linux – Windows y otros), un servidor **Python** es la mejor opción, viene preinstalado en casi todas las versiones de Linux y levantar el servicio toma dos segundos con el comando `python3 -m http.server 8080`.

El de **npm** lo reservaríamos únicamente si algún día estamos programando una página web con javascript y necesitamos probar funciones avanzadas de navegador.

QUESTION 2

Find a way to start a simple HTTP server inside Pwnbox or your local VM using "php". Submit the command that starts the web server on the localhost (127.0.0.1) on port 8080.

Esta es otra pregunta que me hizo ir a internet porque hasta ese momento cuando quería levantar un servidor utilizando php, yo pensaba automáticamente en Apache.

Lo primero es recordar un concepto básico que a algunos se les olvida y piensan que php es solo un lenguaje para crear páginas web "viejas" como las de wordpress, cuando en realidad PHP es un lenguaje interpretado al igual que Python. A diferencia de este último, php no siempre viene preinstalado en Linux. Si lo instalamos, llamamos al ejecutable con php en la terminal. De ahí la primera parte de la respuesta.

En este caso, colocamos en la terminal **php -h** y revisamos las opciones podemos ver como la opción **-S** es la que nos puede ayudar.

```
-H      Hide any passed arguments from external tools.  
-S <addr>:<port> Run with built-in web server.  
-t <docroot>    Specify document root <docroot> for built-in web server.  
-s      Output HTML syntax highlighted source.  
-v      Version number
```

Dado que nos dan la dirección ip y el número de puerto con la que debemos levantar ese servidor, la respuesta queda así:

php -S 127.0.0.1:8080

Question 2

XP +20 ^

Find a way to start a simple HTTP server inside Pwnbox or your local VM using "php". Submit the command that starts the web server on the localhost (127.0.0.1) on port 8080.

php -S 127.0.0.1:8080



¿Pero porque no ejecutar Apache en vez de este servidor php?

<https://www.php.net/manual/es/features.commandline.webserver.php>

Servidor web interno

Advertencia Este servidor web está destinado a ayudar en el desarrollo de aplicaciones. También puede ser útil para pruebas y para demostraciones de aplicaciones que se ejecutan en entornos controlados. Sin embargo, no está diseñado para ser un servidor web completo. Por lo tanto, no debe utilizarse en una red pública.

La razón es muy sencilla, el servidor que se levanta con `php -S` es un servidor singlethreaded (monohilo), lo que quiere decir que solo atiende a una solicitud a la vez; por ejemplo, si yo estoy en el servidor, mi amigo no podrá ingresar y mucho menos múltiples clientes. Por eso se utilizan servidores como Apache o Nginx para producción real.



File System Management

Question 1

How many partitions exist in our Pwnbox? (Format: 0)

No requiere mucha explicación: `lsblk`

```
[us-academy-6]-[10.10.14.246]-[htb-ac-2690832@htb-sqkle8engw]-[~]
[★]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
vda         254:0    0   50G  0 disk
├─vda1      254:1    0  47.4G  0 part /
├─vda2      254:2    0    1K  0 part
└─vda5      254:5    0   2.6G  0 part [SWAP]
[us-academy-6]-[10.10.14.246]-[htb-ac-2690832@htb-sqkle8engw]-[~]
[★]$ lsblk | grep part
├─vda1      254:1    0  47.4G  0 part /
├─vda2      254:2    0    1K  0 part
└─vda5      254:5    0   2.6G  0 part [SWAP]
[us-academy-6]-[10.10.14.246]-[htb-ac-2690832@htb-sqkle8engw]-[~]
[★]$ lsblk | grep part | wc -l
3
```

Y la respuesta es

Question 1 KP +20 ^

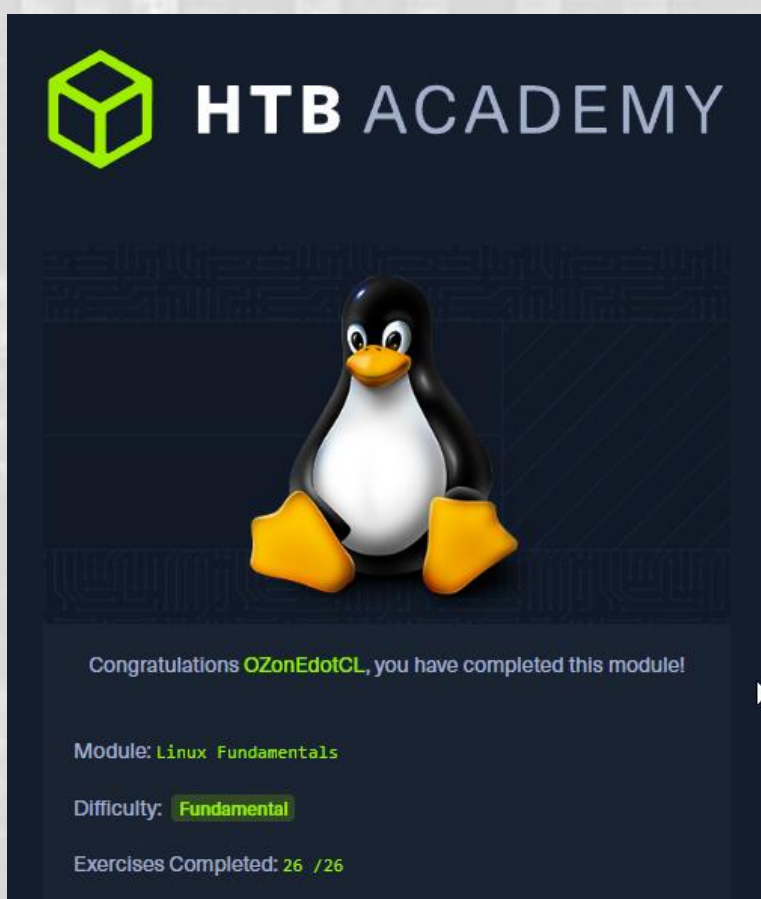
How many partitions exist in our Pwnbox? (Format: 0)

✓

Conclusión

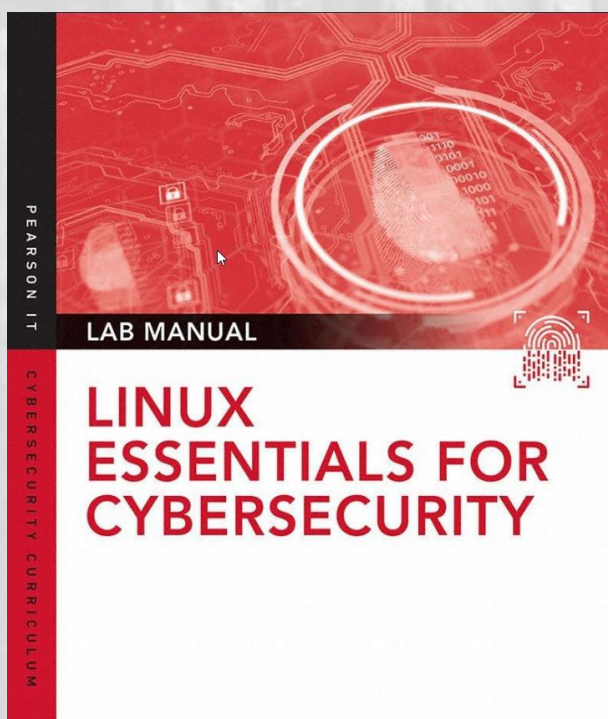
¿Hemos terminado?

Si completaste todas las preguntas, hiciste las actividades extra, entendiste el material y buscaste caminos alternativos para llegar a las respuestas... entonces sí, sin duda terminaste la primera parte de tu camino por Linux. Habiendo estudiado otros recursos anteriormente, creo que este curso va directo al grano: te obliga a pensar, y ese desafío es el que realmente te enseña.



¿Qué viene ahora?

Desde mi posición, sin ser un experto en Linux y compartiendo solo una ruta posible, creo que una excelente alternativa para quienes nos interesa la ciberseguridad es hacer laboratorios que nos permitan practicar y aplicar lo aprendido. En este sentido, el siguiente libro me parece una opción ideal para dar el próximo paso:



Linux Essentials for Cybersecurity- William "Bo" Rothwell

Por lo tanto, si volvemos a la pregunta inicial de si hemos terminado, la respuesta más obvia es: **esto recién comienza**.

La lógica que aplico para entender, aprender y resolver problemas no tiene por qué ser la misma que la tuya. De hecho, me encantaría ver más publicaciones de la comunidad mostrando su propio proceso y compartiendo conocimientos que, de seguro, muchos todavía no conocemos.